python

Yair Mau

Table of contents

Home **5**
- Graph parameters . . . 6
- Fitzhugh-Nagumo — Labyrinthine Patterns . . . 6
- Conway's Game of Life, acorn initial condition . . . 7
- Least squares fit of nonlinear function . . . 7
- Fun with histograms . . . 8
- Fancy subplot grid . . . 9
- Streamplot . . . 10
- Palette . . . 11
- The time dependent Ginzburg-Landau equation . . . 12
- The double pendulum . . . 12
- The Hilbert curve . . . 13
- A hysteresis mechanism . . . 13
- Contour plot . . . 13
- Least action principle on the beach . . . 14
- This website's logo . . . 15
- Bars . . . 16

1 graph parameters **19**

2 Fitzhugh-Nagumo Equation **22**
- 2.1 Introduction . . . 22
- 2.2 The code . . . 22
  - 2.2.1 import packages . . . 22
  - 2.2.2 define class with all the methods . . . 22
  - 2.2.3 run simulation, save snapshots . . . 25
  - 2.2.4 make movie, delete snapshots . . . 26
- 2.3 The semi-spectral method . . . 26
  - 2.3.1 the method . . . 27
  - 2.3.2 step by step . . . 28
  - 2.3.3 example . . . 29

3 Conway's Game of Life **30**
- 3.1 Introduction . . . 30
- 3.2 The code . . . 30

4 Least Squares **32**
4.1 Introduction . . . 32
4.2 The code . . . 33

5 Fun with histograms **36**
5.1 Introduction . . . 36
5.2 The code . . . 37

6 Fancy subplot grid **41**
6.1 Introduction . . . 41
6.2 The code . . . 42

7 Streamplot **48**
7.1 Introduction . . . 48
7.2 The code . . . 49

8 Palette **53**

9 The time-dependent Ginzburg-Landau equation **59**
9.1 Introduction . . . 59
9.2 Code . . . 59

10 Double Pendulum **62**
10.1 Introduction . . . 62
10.2 The code . . . 62

11 Hilbert Curve **66**
11.1 Introduction . . . 66
11.2 The code . . . 66

12 A hysteresis mechanism **69**
12.1 Introduction . . . 69
12.2 The code . . . 69

13 Contour plot **74**

14 Best path **78**

15 Logo **84**
15.1 Equation for an offset parabola . . . 84
15.2 The standard logo . . . 84
15.3 Logo in white . . . 86
15.4 Annotated logo . . . 87
15.5 Chunky logo to be converted into favicon . . . 89
15.6 Duotone . . . 90

15.7 Variation on the theme 91
15.8 logo with URL 92

16 Bar plot **94**
16.1 Introduction 96
16.2 The code 96

Home

Python is a great language for scientific computing, most of the programming done by our group is in python. We provide below some links for learning this language, and below we offer many python code examples. You are invited to download these codes, tweak with them, break them, hack them as you wish!

Most codes focus on plotting, but other algorithms such as numerical integration and Fourier transforms can also be found.

Some useful links for learning python:

- Python Data Science Handbook
- Learn Python the Hard Way
- A Crash Course in Python for Scientists
- Software Carpentry
- Fun with Lists
- Python | Codecademy
- Scipy Lecture Notes
- A gallery of interesting IPython Notebooks
- Think Python: How to Think Like a Computer Scientist

- The Python Graph Gallery

This might be overwhelming, so I suggest you to follow this:

My advice on learning Python:Don't set out to ”learn Python”. Choose a problem you're interested in and learn to solve it with Python.

— Jake VanderPlas (@jakevdp) September 10, 2017

Start by downloading Anaconda, a package manager application that will help you get started with python in all platforms.

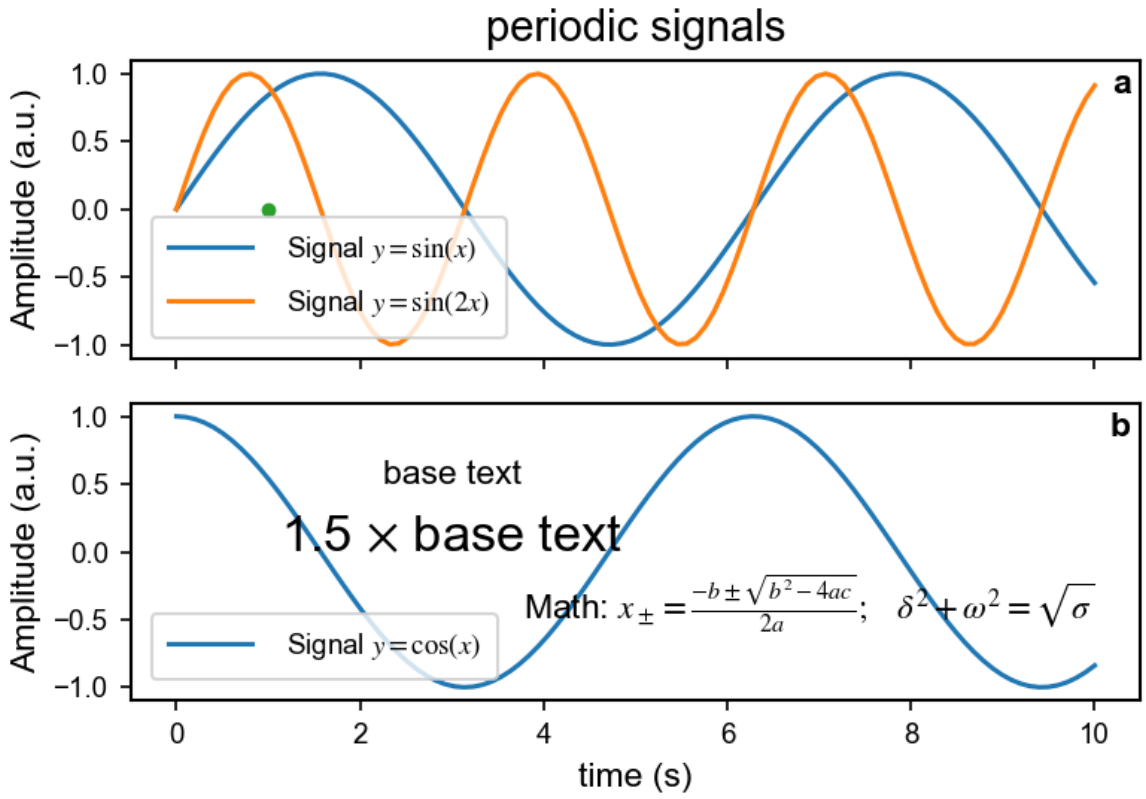
Make sure you are acquainted with ipython (interactive python, try it here), and with Jupyter notebook.

I recommend Sublime Text for writing code, and you can install the Anaconda package to it to have a smooth IDE. [not the same anaconda as mentioned above! Yes, it's confusing, I know.]

Graph parameters

Jupyter notebook

Main features: The `plot_style.py` file defines parameters for nice-looking graphs.



Fitzhugh-Nagumo — Labyrinthine Patterns

Jupyter notebook

Main features: how to make a movie, time-integration methods (semi-spectral and Euler)

Conway's Game of Life, acorn initial condition

Jupyter notebook

This is a (slightly) modified version of Glowing Python's code. I make it available here because it features a few nice things:

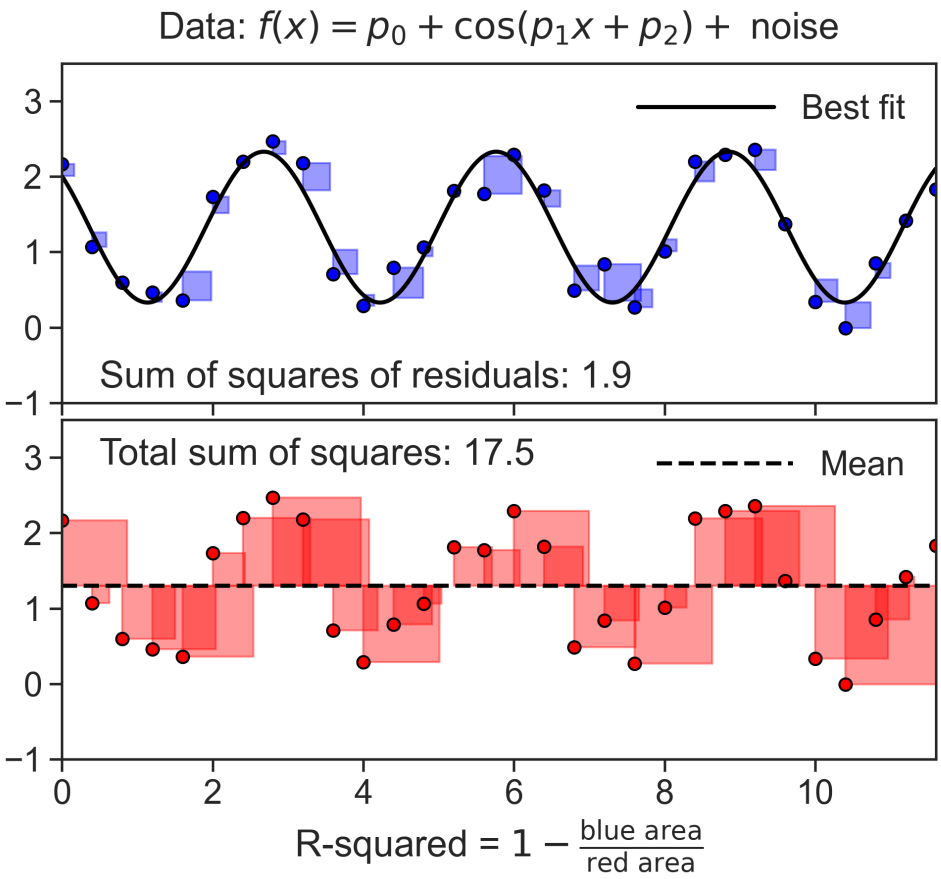
- how to make a movie using matplotlib.animation
- how to write a generator (function with yield)
- how to plot a sparce array (spy)

Main features: `matplotlib.animation, yield, with, matplotlib.pyplot.spy`

Least squares fit of nonlinear function

Jupyter notebook

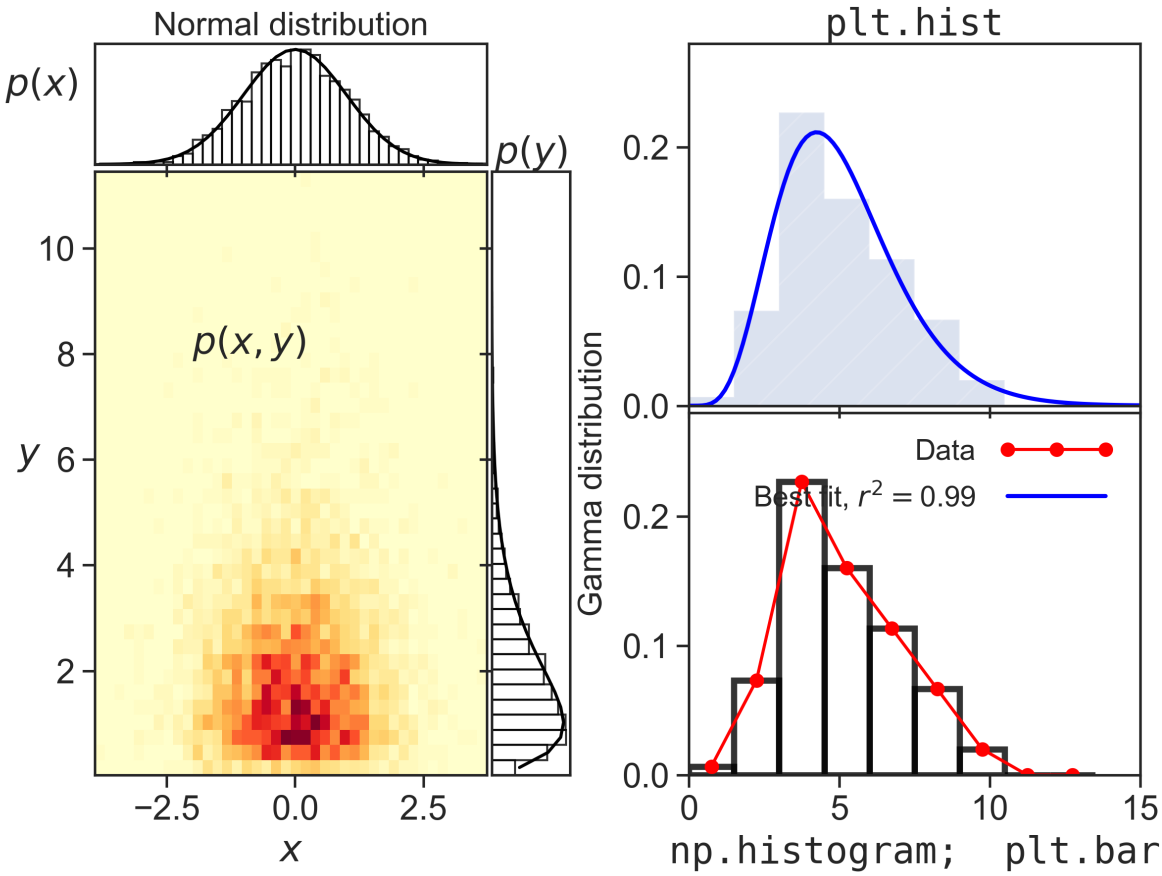
Main features: `LaTeX text, scipy.optimize.curve_fit, matplotlib.patches`



Fun with histograms

Jupyter notebook

Main features: `np.histogram`, `plt.hist`, `plt.bar`, `plt.barh`, `gridspec`, least squares fit of nonlinear function, `plt.hist2d`

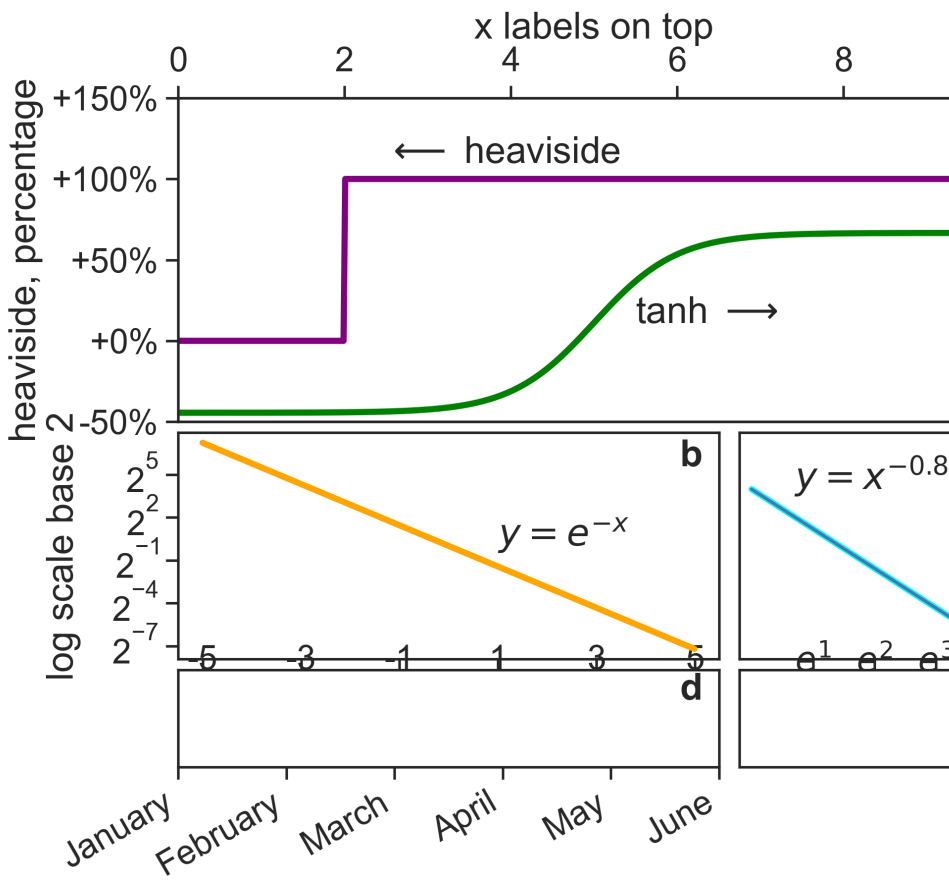


Fancy subplot grid

Jupyter notebook

Highly customizable subplot structure. Also, figure contains several axis configurations and labeling options.

Main features: `gridspec subplots; numpy-compatible heaviside; label, ticks and`

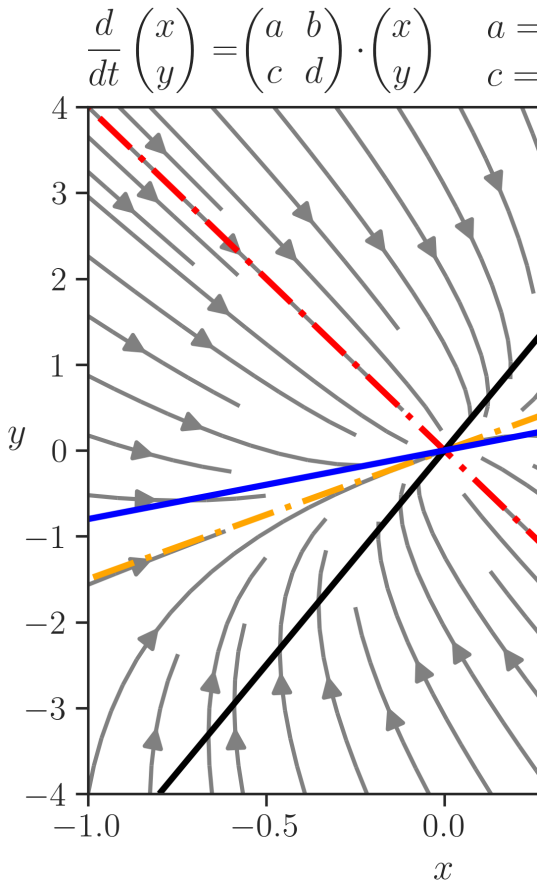


`axis manipulations; log scale`

Streamplot

Jupyter notebook

Streamplot of a two-dimensional linear system, with eigenvectors and nullclines. Python shows LaTeX equations beautifully.



Main features: `meshgrid`, `streamplot`, `contour`, `legend`, `LaTeX`

Palette

Jupyter notebook

Given a named color, returns palettes with varying hue, lightness, and saturation.



The time dependent Ginzburg-Landau equation

Jupyter notebook

Numerical integration of a parabolic partial differential equation, using finite differences: Euler step to advance time, and a 5-point stencil to approximate the Laplacian.
Main features: `imshow, colorbar, set_data`

The double pendulum

Jupyter notebook

Numerical integration of the equations of motion of the double pendulum. This time, scipy's ode itegrator was used. Nice example of how to make a movie.
Main features: `scipy.integrate.ode, set_data, set_aspect('equal'), remove plot, movie`

The Hilbert curve

Jupyter notebook

Construction of the Hilbert curve as a Lindenmayer system (L-system).
Main features: `string operations, movie`

A hysteresis mechanism

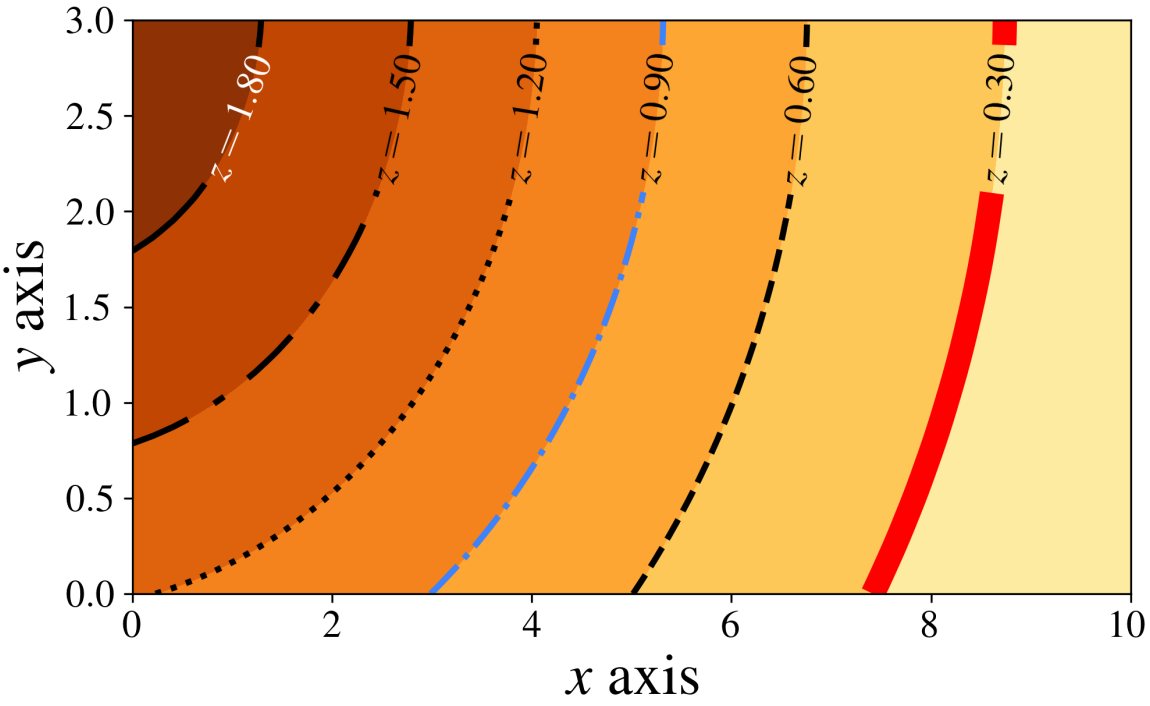
Jupyter notebook

Hysteresis mechanism created by the bistability of states. System goes to minimum points u in the energy functional $f = u^4 - 2u^2 + hu$. The parameter h is ramped down and up during this simulation.
Main features: `sympy analytical calculations, numpy dtypes, movie`

Contour plot

Jupyter notebook

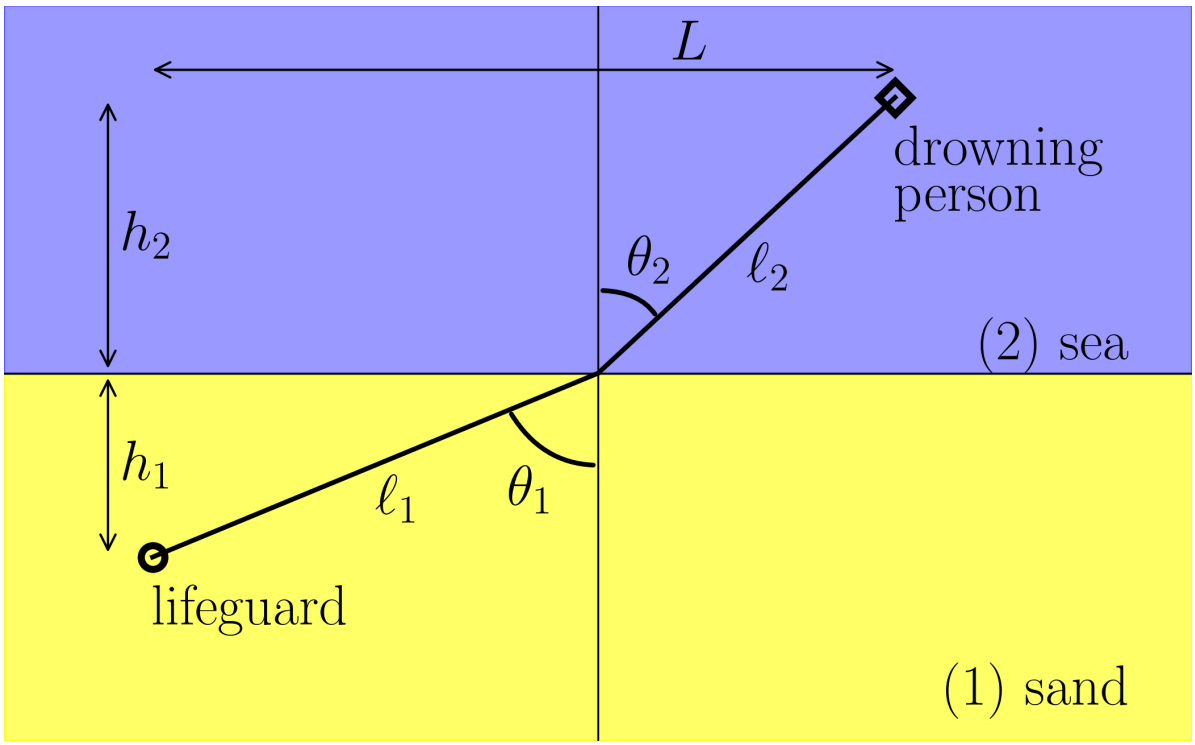
Contour plot with many customizable options. Also, a nice way to truncate a colormap so it gives the color range that you want.
Main features: `truncate_colormap, contour, contourf (fill), clabel (contour label)`



Least action principle on the beach

Jupyter notebook

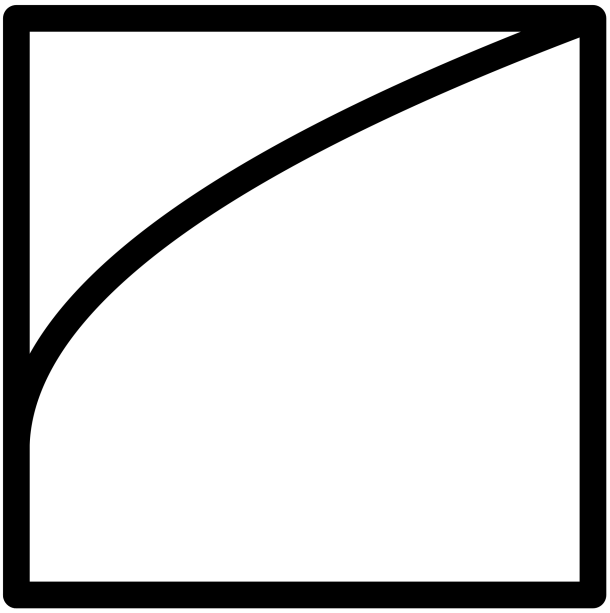
How should lifeguard run in order to save a drowning person in minimal time? Answer: by using Snell's law of refraction!
This is a nice example how to use spines (x and y axis form a cross), instead of rectangular figures as usual. Also, "annotations" are used, where things can be labeled with the help of arrows.
Main features: `spines, matplotlib.patches.Rectangle, annotate`



This website's logo

Jupyter notebook

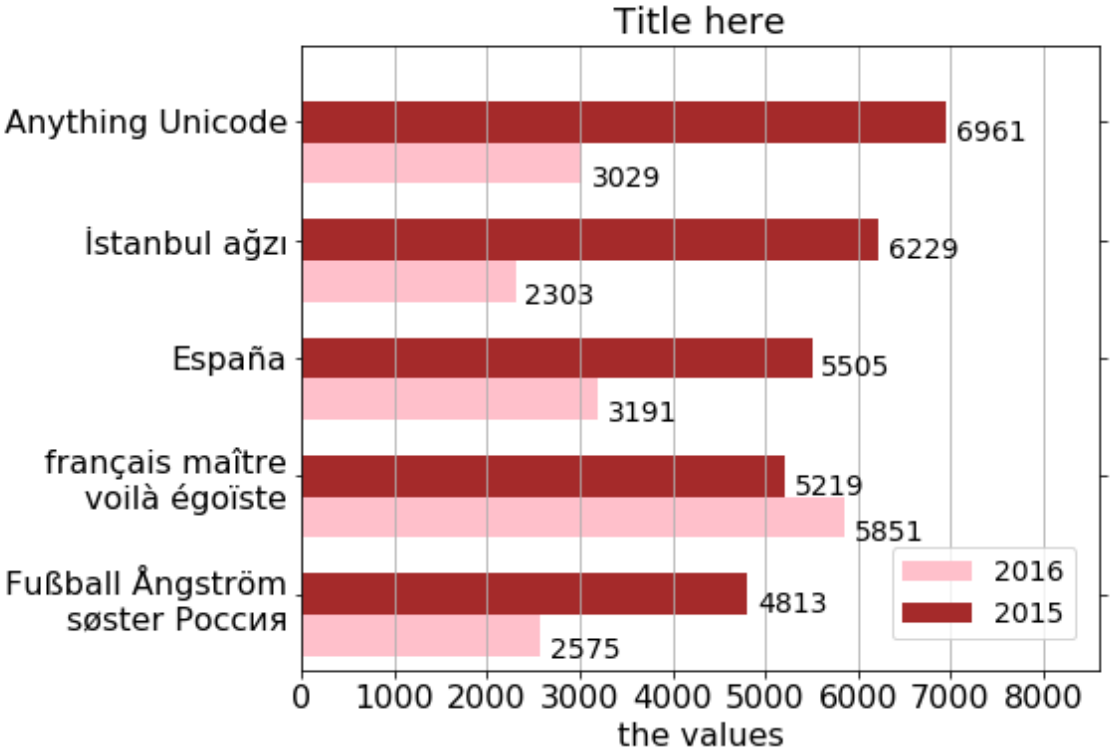
Simple example of how to make a figure without any visible axes.
Main features: `set_axis_off, fill_between, matplotlib inline plot on Jupyter`

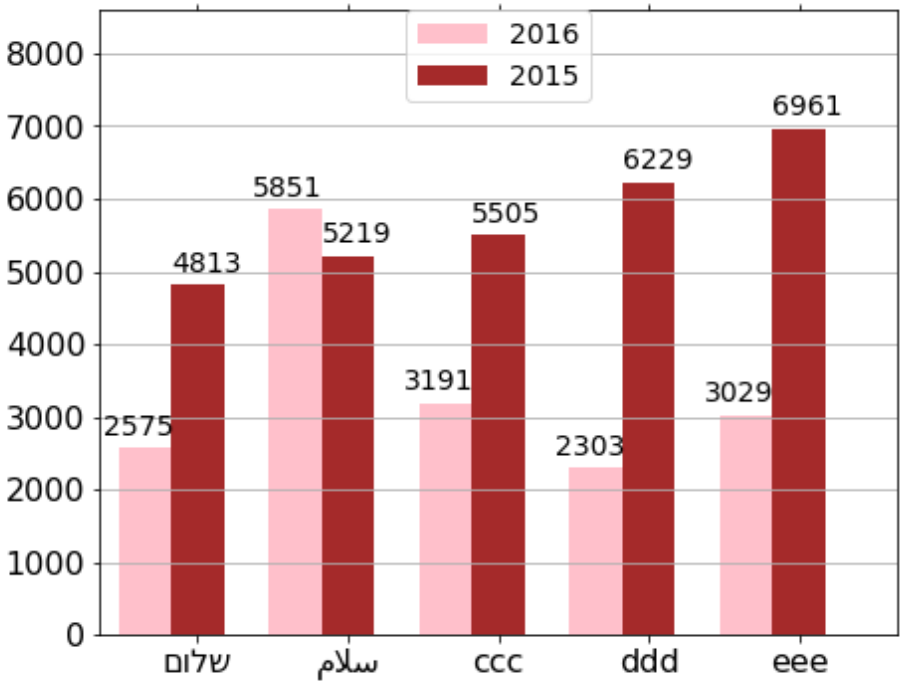


Bars

Jupyter notebook

Horizontal and vertical bars, with numeric legends. Unicode support.
Main features: `unicode, bar, barh, grid`


Title here
Anything Unicode
İstanbul ağzı
España
français maître voilà égoïste
Fußball Ångström søster Россия
6961
3029
6229
2303
5505
3191
5219
5851
4813
2575
2016
2015
0
1000
2000
3000
4000
5000
6000
7000
8000
the values


2016
2015
8000
7000
6000
5000
4000
3000
2000
1000
0
6961
6229
5851
5505
5219
4813
3191
3029
2575
2303
שלום
سلام
ccc
ddd
eee

"The comments aren't there to explain the code to the programmers, the code is there to explain the comments to the computer."
—Andy Harris, How to begin thinking like a programmer

1 graph parameters

import libraries:

```
import matplotlib.pyplot as plt
import numpy as np
```

Download the file plot_style.py.

Explanation:

- width_fraction: default 1.0. Assume the generated image will be included in a latex document. If you want this image to occupy 90% of the width of the text, then you will write something like `\includegraphics[width=0.9\textwidth]{my_figure.pdf}`. In this case use width_fraction=0.9. If you don't need any of this, just choose `1.0`.
- view_scale: default 1.0. Often, the figure generated looks too small on a jupyter notebook, and it's hard to know if everything is in the right place where we wanted. While developing the figure, you can make it look larger, e.g. making it twice as large by typing `view_scale=2.0`. Once you're satisfied with the image, and you're ready to produce the final pdf version, make it the intended size using `view_scale=1.0`.
- columns: default 1. Will this figure be embeded in a 1-column latex document, or a 2-column latex document?
- height_ratio: default 0.618. This is the height dimension relative to the width.

```
from plot_style import set_pub_style
style = set_pub_style(width_fraction=0.9, columns=1, view_scale=1.5, height_ratio=0.7)
# metadata = set_pub_style(width_fraction=0.9, columns=1, view_scale=2.0, height_ratio=0.7)

# this is needed to prevent the figure from being cropped when shown in a Jupyter notebook.
# what we see on the screen is the same as what we get in the saved PDF file.
%config InlineBackend.print_figure_kwargs = {"bbox_inches": None}

fig, ax = plt.subplots(2, 1, sharex=True)
fig.subplots_adjust(left=0.13, right=0.98, top=0.90, bottom=0.13, hspace=0.15, wspace=0.02,)

x = np.linspace(0, 10, 100)
ax[0].plot(x, np.sin(x), label=r"Signal $y = \sin(x)$")
```

```
ax[0].plot(x, np.sin(2*x), label=r"Signal $y = \sin(2x)$")
ax[1].plot(x, np.cos(x), label=r"Signal $y = \cos(x)$")

ax[0].plot([1], [0], marker="o")
ax[1].text(3, 0.5, r"base text", ha="center")
# use the `style` dictionary to modify a parameter.
ax[1].text(3, 0.0, r"1.5 $\times$ base text", ha="center", fontsize=1.5 * style["font"]["base
ax[1].text(10, -0.5, r"Math: $x_\pm=\frac{-b\pm\sqrt{b^2-4ac}}{2a};\quad \delta^2+\omega^2=\

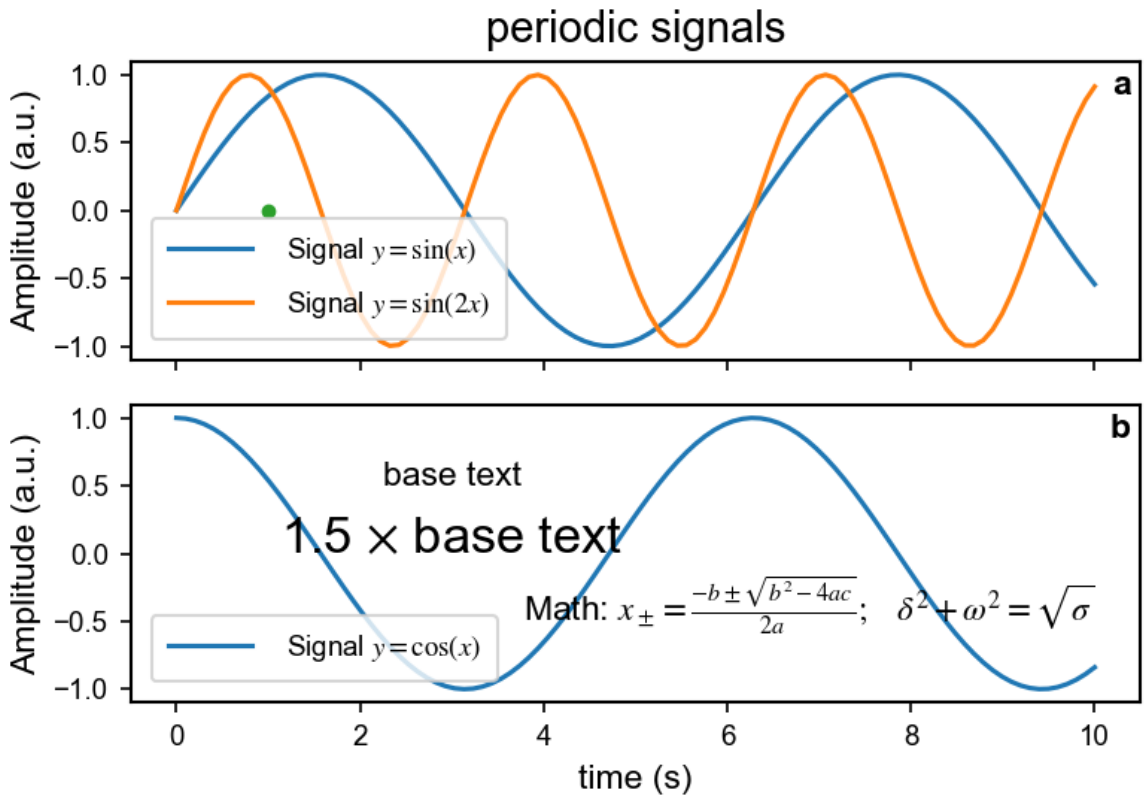
ax[0].set_ylabel("Amplitude (a.u.)")
ax[0].legend()

ax[1].set_xlabel("time (s)")
ax[1].set_ylabel("Amplitude (a.u.)")
ax[1].legend(loc="lower left")

ax[0].text(0.99, 0.97, r"a", transform=ax[0].transAxes,
         horizontalalignment='right', verticalalignment='top',
         fontweight="bold")
ax[1].text(0.99, 0.97, r"b", transform=ax[1].transAxes,
         horizontalalignment='right', verticalalignment='top',
         fontweight="bold")

ax[0].set(title="periodic signals")
# fig.suptitle("periodic signals")

fig.savefig(
    "stam_graph.pdf",
    format="pdf",
    metadata=style["metadata"],
    bbox_inches=None,
)
fig.savefig(
    "stam_graph.png",
    format="png",
    bbox_inches=None,
)
```


periodic signals
a
b
Amplitude (a.u.)
1.0
0.5
0.0
−0.5
−1.0
Signal $y = \sin(x)$
Signal $y = \sin(2x)$
base text
1.5 × base text
Math: $x_{\pm} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$; $\delta^2 + \omega^2 = \sqrt{\sigma}$
Signal $y = \cos(x)$
0
2
4
6
8
10
time (s)

2 Fitzhugh-Nagumo Equation

2.1 Introduction

We simulate the Fitzhugh-Nagumo equations

$$u_t = u - u^3 - v + \nabla^2 u v_t = \epsilon(u - a_1 v - a_0) + \delta \nabla^2 v,$$

using the semi-spectral time integration method.

This simultation was heavily inspired by Aric Hagberg's simulation in "From Labyrinthine Patterns to Spiral Turbulence", PRL 1994.

The code below provides 3 initial conditions, "squiggle, blocks, and random". For time integration, besides the spectral method, we also provide the Euler method. Details about the semi-spectral method can be found after the code.

Parameters: $\epsilon = 0.3$, $\delta = 2.0$, $a_1 = 1.4$, and $a_0 = 0$.

Other simulations and Python examples can be found on my website: yairmau.com.

2.2 The code

2.2.1 import packages

```
import numpy as np
import matplotlib.pyplot as plt
import os
from matplotlib import rcParams
rcParams['font.family'] = 'monospace'
```

2.2.2 define class with all the methods

```
class FitzHughNagumo(object):
    def __init__(self, epsilon=0.3, delta=2.0, a1=1.4, a0=0.0,
                 n=(256, 256), l=(400, 400),
                 start=0.0, step=1.0, finish=2000.0, dt=0.1,
                 integration_type="spectral"):
        self.epsilon = epsilon
        self.delta = delta
        self.a1 = a1
        self.a0 = a0
        self.n = n
        self.l = l
        self.start = start
        self.step = step
        self.finish = finish
        self.dt = dt
        self.integration_type = integration_type
        self.rhs_a = np.zeros((2, self.n[0], self.n[1]))

    def spectral_multiplier(self):
        dx = float(self.l[0]) / self.n[0]
        dy = float(self.l[1]) / self.n[1]
        # wave numbers
        fx = 2.0 * np.pi * np.fft.fftfreq(self.n[0], dx)
        fy = 2.0 * np.pi * np.fft.fftfreq(self.n[1], dy)
        kx = np.outer(fx, np.ones(self.n[0]))
        ky = np.outer(np.ones(self.n[1]), fy)
        # multiplier
        mult_a = np.zeros((2, self.n[0], self.n[1]))
        mult_a[0] = np.exp(-(kx ** 2 + ky ** 2) * self.dt)  # u
        mult_a[1] = np.exp(-self.delta * (kx ** 2 + ky ** 2) * self.dt)  # v
        return mult_a

    def rhs_reaction(self, a):
        u = a[0]  # alias
        v = a[1]  # alias
        # FHN right hand side
        self.rhs_a[0] = u - u ** 3 - v
        self.rhs_a[1] = self.epsilon * (u - self.a1 * v - self.a0)
        return self.rhs_a

    def rhs_euler(self, a):
```

```
        # boundary conditions in laplacian
        laplacian = self.periodic_laplacian
        u = a[0]  # alias
        v = a[1]  # alias
        dx = float(self.l[0]) / self.n[0]
        # FHN right hand side
        self.rhs_a[0] = u - u ** 3 - v + laplacian(u, dx=dx)
        self.rhs_a[1] = self.epsilon * (u - self.a1 * v - self.a0) + \
                        self.delta * laplacian(v, dx=dx)
        return self.rhs_a

    def draw(self, a, t):
        u = a[0]
        self.im = plt.imshow(u.real, cmap="Greys_r", origin='lower',
                             vmin=-0.534522, vmax=0.534522,
                             interpolation="gaussian")
        self.title = plt.title('time = {:>4.0f}'.format(0))
        plt.xticks([])
        plt.yticks([])
        self.im.figure.canvas.draw()

    def draw_update(self, a, t):
        u = a[0]
        self.title.set_text('time = {:>4.0f}'.format(t))
        self.im.set_data(u.real)
        self.im.figure.canvas.draw()

    def save_frame(self, i):
        fname = "_tmp{:05d}.png".format(i)
        self.frame_names.append(fname)
        self.fig.savefig(fname, bbox_inches='tight', dpi=300)

    def periodic_laplacian(self, u, dx=1):
        """Return finite difference Laplacian approximation of 2d array.
        Uses periodic boundary conditions and a 2nd order approximation."""
        laplacian = (np.roll(u, -1, axis=0) +
                     np.roll(u, +1, axis=0) +
                     np.roll(u, -1, axis=1) +
                     np.roll(u, +1, axis=1) - 4.0 * u) / (dx ** 2)
        return laplacian

    def random_ic(self):
```

```
        return 0.5 * (np.random.random((2, self.n[0], self.n[1])) - 0.5)

    def blocks_ic(self):
        a = np.ones((2, self.n[0], self.n[1]))
        a[0] = 0.534522
        a[1] = 0.381802
        n = self.n
        p = n[0] / 8
        a[0][3 * p - 4:3 * p + 4, 5 * p - 4:5 * p + 4] = -0.534522
        a[0][6 * p - 4:6 * p + 4, 3 * p - 4:3 * p + 4] = -0.534522
        return a

    def squiggle_ic(self):
        a = np.ones((2, self.n[0], self.n[1]))
        l = self.l
        uplus = 0.534522
        vplus = 0.381802
        uminus = -uplus
        X, Y = np.meshgrid(np.linspace(0, self.l[0], self.n[0]),
                           np.linspace(0, self.l[0], self.n[0]))
        cos_term = 0.05 * l[0] * np.sin(10 * (2 * np.pi) * Y / l[1] + np.pi * 0.3)
        exp_term = np.exp(-((Y - l[1] / 2) / (0.1 * l[1])) ** 2)
        width = 0.05 * l[0]
        Z = np.exp(-((X - l[0] / 2 + cos_term * exp_term) / width) ** 2)
        a[0] = uplus
        a[1] = vplus
        a[0][Z > 0.8] = uminus
        return a
```

2.2.3 run simulation, save snapshots

```
plt.ion()
plt.clf()
foo = FitzHughNagumo()
foo.fig = plt.figure(1)
ax = foo.fig.add_subplot(111)
a = foo.squiggle_ic()
mult_a = foo.spectral_multiplier()
fft_a = np.fft.fftn(a, axes=(1, 2))
t = foo.start
```

```
foo.draw(a, t)
foo.frame_names = []
foo.save_frame(0)
for i, tout in enumerate(np.arange(foo.start + foo.step,
                                   foo.finish + foo.step,
                                   foo.step)):
    while t < tout:
        if foo.integration_type == "spectral":
            rhs_a = foo.rhs_reaction(a)
            fft_a = mult_a * (fft_a + foo.dt * np.fft.fftn(rhs_a, axes=(1, 2)))
            a = np.fft.ifftn(fft_a, axes=(1, 2))
        if foo.integration_type == "euler":
            a = a + foo.dt * foo.rhs_euler(a)
        t += foo.dt
    foo.draw_update(a, t)
    foo.save_frame(i + 1)
```

2.2.4 make movie, delete snapshots

```
fps = 24
frames = "_tmp%5d.png"
movie_command = "ffmpeg -y -r {:} -i {:} fhn.mp4".format(fps, frames)
os.system(movie_command)
for fname in foo.frame_names:
    os.remove(fname)
```

2.3 The semi-spectral method

The explanation below was taken from my thesis: "Pattern Formation in Spatially Forced Systems: Application to Vegetation Restoration".

The semi-spectral method is extremely useful when working with reaction-diffusion systems, and with parabolic PDEs in general. This was the method used to run all the simulations of the Swift-Hohenberg model in this thesis, and it proved to be reliable and fast. The explanation below is a summary of "Spectral algorithms for reaction-diffusion equations", by Richard V. Craster and Roberto Sassi, with a step by step recipe, so the reader can easily apply the method to any suitable problem.

2.3.1 the method

The semi-spectral transform method is very useful when we have to integrate a system that evolves really slowly. Let us say we have a (parabolic) system of the form:

$$u_t = \epsilon u + f(u) + D\nabla^2 u, \tag{1}$$

where $f(u)$ is a nonlinear function. First, we compute the Fourier transform of (**??**):

$$\hat{u}_t = \epsilon \hat{u} + \hat{f}(u) - k^2 D\hat{u}, \tag{2}$$

where the hat denotes the Fourier transform.

We rearrange (**??**) in the following way:

$$\hat{u}_t + a\hat{u} = \hat{f}(u), \tag{3}$$

where $a = -\epsilon + k^2 D$, and now we make a variable substitution

$$\hat{v}(k,t) = \hat{u}(k,t)\, e^{at} \tag{4a}$$
$$\hat{v}_t = \hat{u}_t e^{at} + a\hat{u}\, e^{at}. \tag{4b}$$

We multiply (**??**) by e^{at} and we finally get

$$\hat{v}_t = e^{at}\hat{f}(u). \tag{5}$$

We can now advance $\hat{v}$ in time using a simple Euler step

$$\hat{v}^{t_{n+1}} = \hat{v}^{t_n} + \Delta t \left(e^{at_n} \hat{f}(u) \right). \tag{6}$$

What we really want is $\hat{u}$, which, according to (**??**), is given by

$$\hat{u}^{t_{n+1}} = \hat{v}^{t_{n+1}} e^{-at_{n+1}} \tag{7a}$$
$$= \hat{v}^{t_{n+1}} e^{-at_n} e^{-a\Delta t} \tag{7b}$$
$$= \left(\hat{v}^{t_n} + \Delta t\; e^{at_n} \hat{f}(u) \right) e^{-at_n} e^{-a\Delta t} \tag{7c}$$
$$= \left(\hat{v}^{t_n} e^{-at_n} + \Delta t\; e^{at_n} \hat{f}(u) e^{-at_n} \right) e^{-a\Delta t} \tag{7d}$$
$$= \left(\hat{u}^{t_n} + \Delta t \hat{f}(u) \right) e^{-a\Delta t}. \tag{7e}$$

There is actually no need to use the variable substitution in (**??**). We now have an expression for $\hat{u}^{t_{n+1}}$:

$$\hat{u}^{t_{n+1}} = \left(\hat{u}^{t_n} + \Delta t \hat{f}(u) \right) e^{-a\Delta t}. \tag{8}$$

Now it is time to go back from the Fourier space to the real space, and for that we use an inverse Fourier transform

$$u^{t_{n+1}} = \mathcal{F}^{-1}[\hat{u}^{t_{n+1}}]. \tag{9}$$

2.3.2 step by step

To implement this technique, one just has to follow the steps below:

- Calculate the Fourier transform of u: $\hat{u} = \mathcal{F}[u]$.
- Have $f(u)$ calculated and then take its Fourier transform: $\hat{f}(u) = \mathcal{F}[f(u)]$.
- For a given lattice with N points, and δx being the distance between them, make the frequency bin vector (matrix) k for your one (two) dimensional system. In python the command would be

```
numpy.fft.fftfreq(N, dx).
```

The frequency bin vector k looks like:

$$k = 2\pi \cdot \left[0, 1, \cdots, \tfrac{N}{2} - 1, -\tfrac{N}{2}, \cdots, -1\right] / (N\,\delta x), \quad \text{if N is even;} \tag{10a}$$

$$k = 2\pi \cdot \left[0, 1, \cdots, \tfrac{N-1}{2}, -\tfrac{N-1}{2}, \cdots, -1\right] / (N\,\delta x), \quad \text{if N is odd.} \tag{10b}$$

Remember that the domain size is given by $L = N\,\delta x$, which means that the denominator in the expressions above can be written simply as L. It is clear from that fact that δk, the tiniest slice of the Fourier space is $\delta k = 2\pi/L$. Corollary: if you want to divide the Fourier space into very many parts, simply have a huge domain.
If the system is two-dimensional, then have k_x and k_y calculated separately. The domain might not be square ($L_x \neq L_y$), and you might want to divide the domain into a different number of points ($N_x \neq N_y$). Anyway, prepare one-dimensional arrays of k_x and k_y as explained above, and then make an *outer product* of these arrays with a *ones* array of length N, as following:

$$k_{x,2d} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \begin{pmatrix} k_{x1} & k_{x2} & \dots & k_{xN} \end{pmatrix} = \begin{pmatrix} k_{x1} & k_{x2} & \dots & k_{xN} \\ k_{x1} & k_{x2} & \dots & k_{xN} \\ & \vdots & & \\ k_{x1} & k_{x2} & \dots & k_{xN} \end{pmatrix} \tag{11}$$

and

$$k_{y,2d} = \begin{pmatrix} k_{x1} \\ k_{x2} \\ \vdots \\ k_{xN} \end{pmatrix} \begin{pmatrix} 1 & 1 & \dots & 1 \end{pmatrix} = \begin{pmatrix} k_{y1} & k_{y1} & & k_{y1} \\ k_{y2} & k_{y2} & \dots & k_{y2} \\ \vdots & \vdots & & \vdots \\ k_{yN} & k_{yN} & & k_{yN} \end{pmatrix}. \tag{12}$$

Then factor $e^{-a\Delta t}$ equals

$$e^{-a\Delta t} = e^{[\epsilon - D(k_x^2 + k_y^2)]\,\Delta t}, \tag{13}$$

where k_x^2 is the element-wise exponentiation of the 2d array $k_{x,2d}$.

- Now that we have all the factors we need, we simply calculate

$$\widehat{u}^{t_{n+1}} = \left(\widehat{u}^{t_n} + \Delta t \widehat{f}(u)\right) e^{[\epsilon - D(k_x^2 + k_y^2)]\,\Delta t}. \tag{14}$$

- We finally go back to the real space by applying the inverse Fourier transform: $u^{t_{n+1}} = \mathcal{F}^{-1}[\widehat{u}^{t_{n+1}}]$.

2.3.3 example

For the parametrically forced Swift-Hohenberg equation

$$\frac{\partial u}{\partial t} = [\epsilon + \gamma \cos(k_f x)]u - u^3 - (\nabla^2 + k_0^2)^2 u, \tag{15}$$

we have

$$f(u) = -u^3 + \gamma u \cos(k_f x), \qquad a = \epsilon - \left(k_0 - k_x^2 - k_y^2\right)^2. \tag{16}$$

3 Conway's Game of Life

3.1 Introduction

This is a (slightly) modified version of Glowing Python's code. I make it available here because it features a few nice things:

- how to make a movie using matplotlib.animation
- how to write a generator (function with yield)
- how to plot a sparce array (`spy`)

3.2 The code

```
import numpy as np
from matplotlib import pyplot as plt
import matplotlib.animation as manimation


def life(X, steps):
    """
     Conway's Game of Life.
     - X, matrix with the initial state of the game.
     - steps, number of generations.
    """
    def roll_it(x, y):
        # rolls the matrix X in a given direction
        # x=1, y=0 left;  x=-1, y=0 right;
        return np.roll(np.roll(X, y, axis=0), x, axis=1)

    for _ in range(steps):
        # count the number of neighbours
        # the universe is considered toroidal
        Y = roll_it(1, 0) + roll_it(0, 1) + \
            roll_it(-1, 0) + roll_it(0, -1) + \
```

```
            roll_it(1, 1) + roll_it(-1, -1) + \
            roll_it(1, -1) + roll_it(-1, 1)
        # game of life rules
        X = np.logical_or(np.logical_and(X, Y == 2), Y == 3)
        X = X.astype(int)
        yield X
```

```
dimensions = (90, 160)  # height, width
X = np.zeros(dimensions)  # Y by X dead cells
middle_y = dimensions[0] / 2
middle_x = dimensions[1] / 2

N_iterations = 600

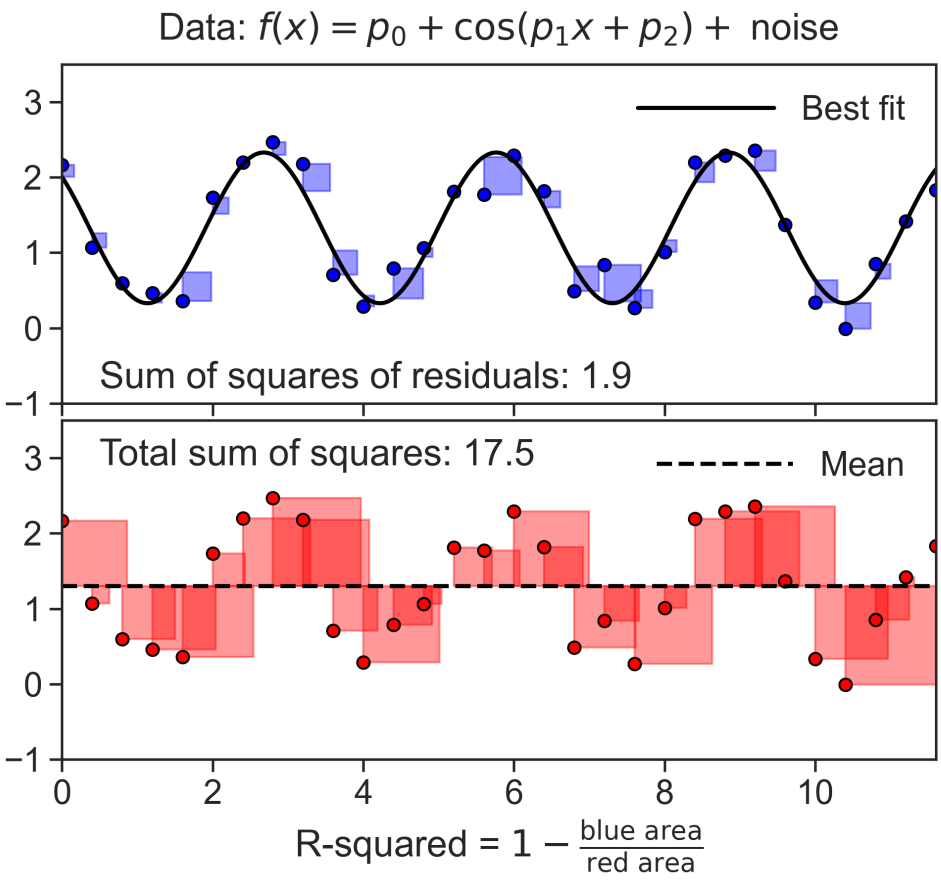
# acorn initial condition
# http://www.conwaylife.com/w/index.php?title=Acorn
X[middle_y, middle_x:middle_x+2] = 1
X[middle_y, middle_x+4:middle_x+7] = 1
X[middle_y+1, middle_x+3] = 1
X[middle_y+2, middle_x+1] = 1
```

```
FFMpegWriter = manimation.writers['ffmpeg']
metadata = dict(title='Game of life', artist='Acorn initial condition')
writer = FFMpegWriter(fps=10, metadata=metadata)

fig = plt.figure()
fig.patch.set_facecolor('black')
with writer.saving(fig, "game_of_life.mp4", 300):  # last argument: dpi
    plt.spy(X, origin='lower')
    plt.axis('off')
    writer.grab_frame()
    plt.clf()
    for i, x in enumerate(life(X, N_iterations)):
        plt.title("iteration: {:03d}".format(i + 1))
        plt.spy(x, origin='lower')
        plt.axis('off')
        writer.grab_frame()
        plt.clf()
```

4 Least Squares

How to fit a nonlinear function to data



4.1 Introduction

This code produces the figure above. It's main tool is the `curve_fit` method, that allows us to fit any function to data, and get optimal parameter values.

4.2 The code

```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.gridspec as gridspec
import scipy.special
from scipy.optimize import curve_fit
import matplotlib.patches as patches
import seaborn as sns
sns.set(style="ticks", font_scale=1.5)
```

Define some functions

```
x = np.arange(0, 12, 0.4)

def func(x, par0, par1, par2):
    return par0 + np.cos(par1 * x + par2)


def add_rec(ax, c, v, col):
    ax.add_patch(
        patches.Rectangle(
            c,          # (x,y)
            np.abs(v),  # width
            v,          # height
            alpha=0.4,
            color=col
        )
    )
```

Now let's plot some stuff

```
%matplotlib notebook

fig, [ax1, ax2] = plt.subplots(2, 1, figsize=(8,6), sharex=True, sharey=True,
                               subplot_kw={'aspect':'equal'}
                              )
fig.subplots_adjust(left=0.04, right=0.98, top=0.93, bottom=0.15,
                    hspace=0.05, wspace=0.02)
params = {#'backend': 'ps',
```

```
          'font.family': 'serif',
          'font.serif': ['Computer Modern Roman'],
          'ps.usedistiller': 'xpdf',
          'text.usetex': True,
          # include here any neede package for latex
          'text.latex.preamble': [r'\usepackage{amsmath}'],
          }
# the parameter values
par = (1.3, 2, 1)
# generating data with noise
y = func(x, *par) + (np.random.random(len(x)) - 0.5)
ax1.plot(x, y, marker='o', ls='None', markerfacecolor="blue", markeredgecolor="black")
ax2.plot(x, y, marker='o', ls='None', markerfacecolor="red", markeredgecolor="black")

# best fit
popt, pcov = curve_fit(func, x, y, p0=(1.5, 1.5, 2.5))  # p0 = initial guess
p0, p1, p2 = popt
# The total sum of squares (proportional to the variance of the data)
SStot = ((y - y.mean()) ** 2).sum()
# The sum of squares of residuals
SSres = ((y - func(x, p0, p1, p2)) ** 2).sum()
Rsquared = 1 - SSres / SStot
# plot best fit
h = np.linspace(x.min(), x.max(), 1001)
fit, = ax1.plot(h, func(h, p0, p1, p2), color='black', linewidth=2)
ax1.legend([fit], ["Best fit"], loc="upper right",
           frameon=False, handlelength=4)
# plot mean
mean, = ax2.plot(h, h * 0 + np.mean(y), ls='--', color='black', linewidth=2)
ax2.legend([mean], ["Mean"], loc="upper right", frameon=False, handlelength=4)

# plot blue and red squares
for ind in np.arange(len(x)):
    x0 = x[ind]
    y0 = y[ind]
    # print(x0,y0)
    v1 = y0 - func(x0, p0, p1, p2)
    v2 = y0 - y.mean()
    add_rec(ax1, (x0, y0), -v1, "blue")
    add_rec(ax2, (x0, y0), -v2, "red")

ax2.text(0.5, 2.9, r"Total sum of squares: {:.1f}".format(SStot))
```

```
ax1.text(0.5, -0.8, r"Sum of squares of residuals: {:.1f}".format(SSres))
# ax2.set_xlabel(
#          r"R-squared = $1 - \frac{\text{blue area}}{\text{red area}}$ = " +
#          "{:.2f}".format(Rsquared))
ax2.set_xlabel(
              r"R-squared = $1 - \frac{\mathrm{blue\ area}}{\mathrm{red\ area}}$"
              )
ax1.set_xlabel(
               r"Data: $f(x) = p_0 + \cos(p_1 x + p_2)+ $ noise ",
               labelpad=12
               )
ax1.xaxis.set_label_position("top")

ax1.set(xlim=[x.min(), x.max()],
        ylim=[-1, 3.5],
        xticklabels=[],
        yticks=np.arange(-1, 4)
       )
ax2.set(
        xticklabels=np.arange(0,13,2)
       )

fig.savefig("least-squares.png", dpi=300)
```

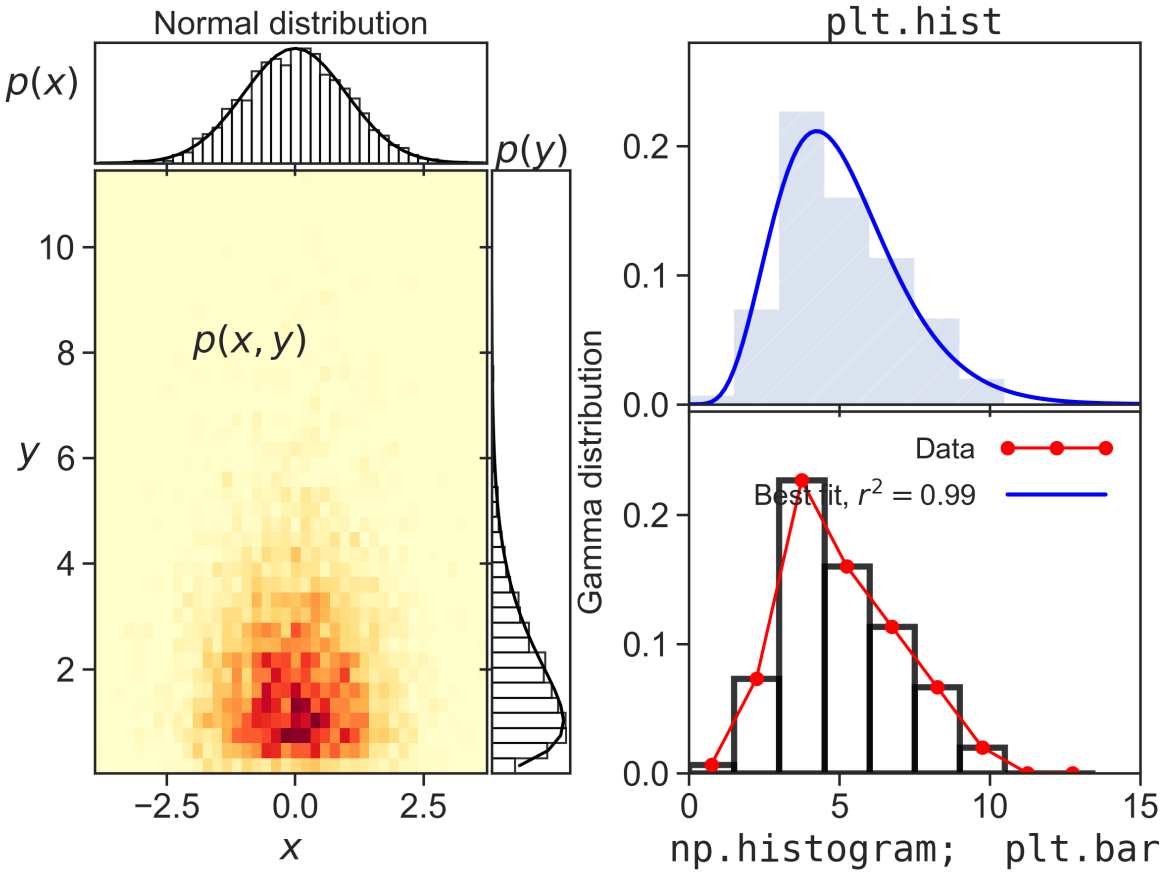
```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.HTML object>
```

```
/Users/yairmau/opt/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:70: UserWarni
```

5 Fun with histograms

1d and 2d histograms



5.1 Introduction

This code produces the figure above. I tried to showcase a few things one can do with 1d and 2d histograms.

5.2 The code

```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.gridspec as gridspec
import scipy.special
from scipy.optimize import curve_fit
import seaborn as sns
sns.set(style="ticks", font_scale=1.5)
```

```
fig = plt.figure(1, figsize=(8,6))  # figsize accepts only inches.

###################################
# Panels on the left of the figure
###################################

gs = gridspec.GridSpec(2, 2, width_ratios=[1, 0.2], height_ratios=[0.2, 1])
gs.update(left=0.10, right=0.50, top=0.95, bottom=0.13,
          hspace=0.02, wspace=0.02)

sigma = 1.0  # standard deviation (spread)
mu = 0.0  # mean (center) of the distribution
x = np.random.normal(loc=mu, scale=sigma, size=5000)
k = 2.0  # shape
theta = 1.0  # scale
y = np.random.gamma(shape=k, scale=theta, size=5000)

# bottom left panel
ax10 = plt.subplot(gs[1, 0])
counts, xedges, yedges, image = ax10.hist2d(x, y, bins=40, cmap="YlOrRd",
                                            density=True)
dx = xedges[1] - xedges[0]
dy = yedges[1] - yedges[0]
xvec = xedges[:-1] + dx / 2
yvec = yedges[:-1] + dy / 2
ax10.set_xlabel(r"$x$")
ax10.set_ylabel(r"$y$", rotation="horizontal")
ax10.text(-2, 8, r"$p(x,y)$")
ax10.set_xlim([xedges.min(), xedges.max()])
ax10.set_ylim([yedges.min(), yedges.max()])
```

```
# top left panel
ax00 = plt.subplot(gs[0, 0])
gaussian = (1.0 / np.sqrt(2.0 * np.pi * sigma ** 2)) * \
    np.exp(-((xvec - mu) ** 2) / (2.0 * sigma ** 2))
xdist = counts.sum(axis=1) * dy
ax00.bar(xvec, xdist, width=dx, fill=False,
         edgecolor='black', alpha=0.8)
ax00.plot(xvec, gaussian, color='black')
ax00.set_xlim([xedges.min(), xedges.max()])
ax00.set_xticklabels([])
ax00.set_yticks([])
ax00.set_xlabel("Normal distribution", fontsize=16)
ax00.xaxis.set_label_position("top")
ax00.set_ylabel(r"$p(x)$", rotation="horizontal", labelpad=20)

# bottom right panel
ax11 = plt.subplot(gs[1, 1])
gamma_dist = yvec ** (k - 1.0) * np.exp(-yvec / theta) / \
    (theta ** k * scipy.special.gamma(k))
ydist = counts.sum(axis=0) * dx
ax11.barh(yvec, ydist, height=dy, fill=False,
          edgecolor='black', alpha=0.8)
ax11.plot(gamma_dist, yvec, color='black')
ax11.set_ylim([yedges.min(), yedges.max()])
ax11.set_xticks([])
ax11.set_yticklabels([])
ax11.set_ylabel("Gamma distribution", fontsize=16)
ax11.yaxis.set_label_position("right")
ax11.set_xlabel(r"$p(y)$")
ax11.xaxis.set_label_position("top")

###################################
# Panels on the right of the figure
###################################

gs2 = gridspec.GridSpec(2, 1, width_ratios=[1], height_ratios=[1, 1])
gs2.update(left=0.60, right=0.98, top=0.95, bottom=0.13,
           hspace=0.02, wspace=0.05)

x = np.random.normal(loc=0, scale=1, size=1000)
y = np.random.gamma(shape=2, size=1000)
```

```
bx10 = plt.subplot(gs2[1, 0])
bx00 = plt.subplot(gs2[0, 0])

N = 100
a = np.random.gamma(shape=5, size=N)
my_bins = np.arange(0,15,1.5)
n1, bins1, patches1 = bx00.hist(a, bins=my_bins, density=True,
                                histtype='stepfilled', alpha=0.2, hatch='/')
bx00.set_xlim([0, 15])
bx00.set_ylim([0, 0.28])
bx00.set_xticklabels([])
bx00.set_xlabel("plt.hist", fontfamily="monospace")
bx00.xaxis.set_label_position("top")

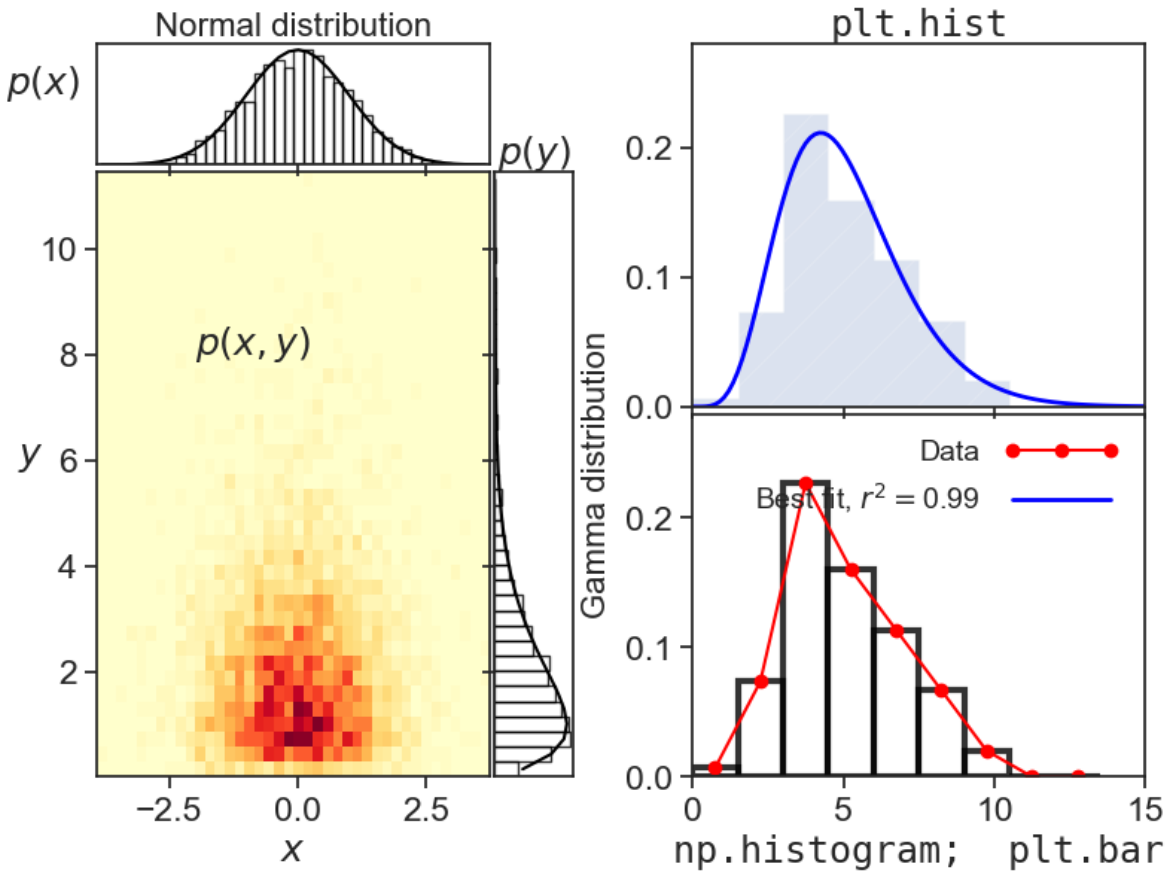
# the following way is equivalent to plt.hist, but it gives
# the user more flexibility when plotting and analysing the results
n2, bins2 = np.histogram(a, bins=my_bins, density=True)
wid = bins2[1] - bins2[0]
red, = bx10.plot(bins2[:-1]+wid/2, n2, marker='o', color='red')
bx10.bar(bins2[:-1], n2, width=wid, fill=False,
         edgecolor='black', linewidth=3, alpha=0.8, align="edge")
bx10.set_xlim([0, 15])
bx10.set_ylim([0, 0.28])
bx10.set_xlabel(r"np.histogram;  plt.bar", fontfamily="monospace")

# best fit

xdata = my_bins[:-1] + wid/2
ydata = n2
def func(x, p1, p2):
    return x ** (p1 - 1.0) * np.exp(-x / p2) / (p2 ** p1 * scipy.special.gamma(p1))
popt, pcov = curve_fit(func, xdata, ydata, p0=(1.5, 1.5))  # p0 = initial guess
p1, p2 = popt
SStot = ((ydata - ydata.mean()) ** 2).sum()
SSres = ((ydata - func(xdata, p1, p2)) ** 1).sum()
Rsquared = 1 - SSres / SStot
h = np.linspace(0,15,101)
bx00.plot(h, func(h, p1, p2), color='blue', linewidth=2)
# dummy plot, just so we can have a legend on the bottom panel
blue, = ax10.plot([100],[100], color='blue', linewidth=2, label="Best fit")
bx10.legend([red,blue],[r'Data',r'Best fit, $r^2=${:.2f}'.format(Rsquared)],
            loc='upper right', frameon=False, handlelength=4,
```

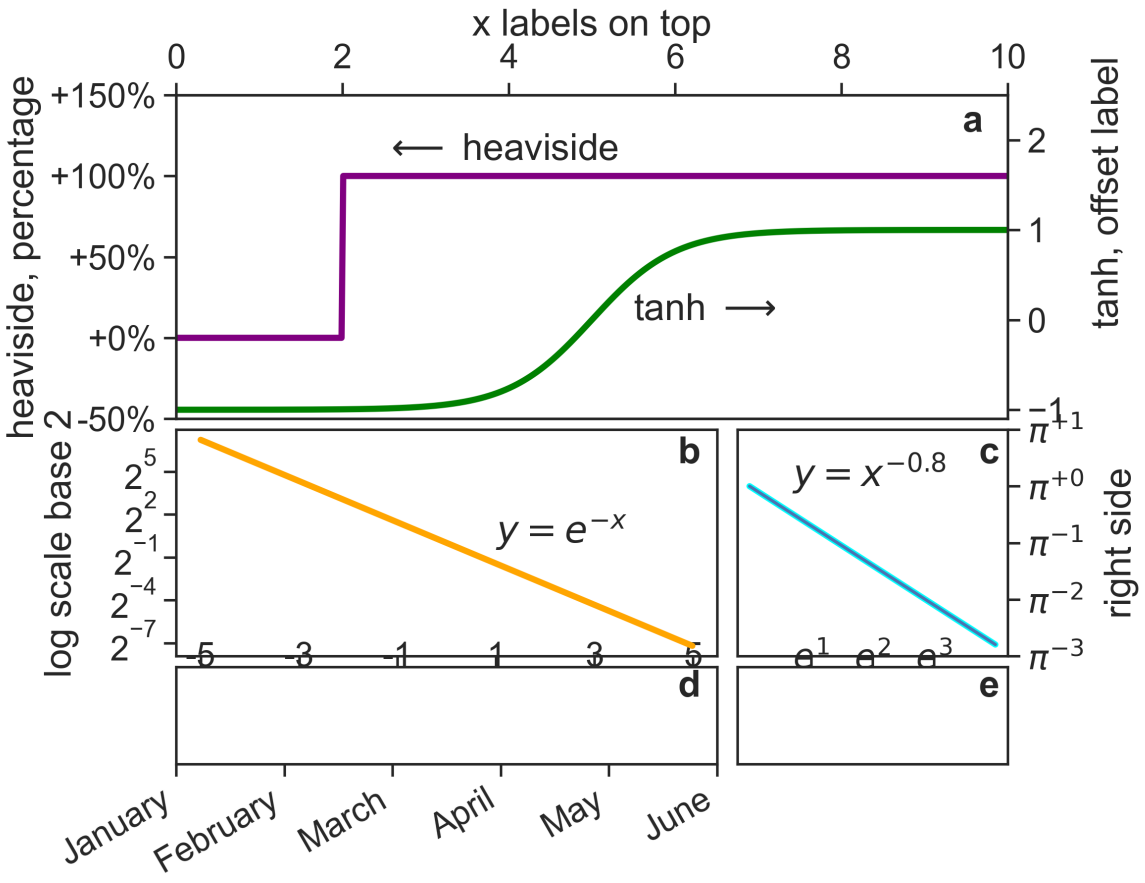
```
            markerfirst=False, numpoints=3, fontsize=14)

fig.savefig("histograms.png",dpi=300)
```



6 Fancy subplot grid

GridSpec is your friend



6.1 Introduction

With GridSpec you can create any combination of panels

6.2 The code

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.ticker import FuncFormatter
import seaborn as sns
sns.set(style="ticks", font_scale=1.5)
```

```
# figsize accepts only inches.
fig = plt.figure(1, figsize=(8, 6))
gs = gridspec.GridSpec(3, 2, width_ratios=[1,0.5], height_ratios=[1,0.7,0.3])
gs.update(left=0.16, right=0.86,top=0.88, bottom=0.13, hspace=0.05, wspace=0.05)

############
# subplot a
############
ax0 = plt.subplot(gs[0, :])

heaviside = lambda x: 0.5 * (np.sign(x) + 1)
x = np.arange(0, 10.01, 0.01)
ax0.plot(x, heaviside(x - 2), color='purple', lw=3)
ax0.text(2.5, 1.1, r"$\longleftarrow$ heaviside")

# y ticks as a percentage
ax0.set_yticks(np.arange(-0.5, 2.0, 0.5))
def to_percent(y, position):
    # Ignore the passed in position. This has the effect of scaling the default
    # tick locations.
    s = "{:+.0f}".format(y * 100)  # str(100 * y)
    # The percent symbol needs escaping in latex
    if matplotlib.rcParams['text.usetex'] is True:
        return s + r'$\%$'
    else:
        return s + '%'
# Create the formatter using the function to_percent. This multiplies all the
# default labels by 100, making them all percentages
formatter = FuncFormatter(to_percent)
# Set the formatter
ax0.yaxis.set_major_formatter(formatter)
```

```
ax0.set_ylabel("heaviside, percentage")

# x ticks on top
ax0.axis([x.min(), x.max(), -0.5, 1.5])
ax0.xaxis.tick_top()
ax0.set_xlabel(r"x labels on top")
ax0.xaxis.set_label_position("top")

# transAxes makes position relative to axes
ax0.text(0.97, 0.97, r"a", transform=ax0.transAxes,
         horizontalalignment='right', verticalalignment='top',
         fontweight="bold")

# copy window with same x axis (y will be different)
ax0b = ax0.twinx()
ax0b.plot(x, np.tanh(x - 5), color="green", linewidth=3)
ax0b.axis([x.min(), x.max(), -1.1, 2.5])
ax0b.text(5.5, 0, r"tanh $\longrightarrow$")
ax0b.set_ylabel(r'tanh, offset label')
ax0b.yaxis.set_label_coords(1.1, 0.70)

############
# subplot b
############
ax10 = plt.subplot(gs[1, 0])

x = np.arange(-5, 5, 0.01)
y = np.exp(-x)
ax10.plot(x, y, color="orange", lw=3)
ax10.set_yscale('log', base=2)
ax10.set_yticks(2.0 ** np.arange(-7, 7, 3))
ax10.text(1.0, 1, r"$y=e^{-x}$")
ax10.set_xticks(np.arange(-5, 6, 2))
ax10.set_xticklabels(np.arange(-5, 6, 2), y=0.15)
ax10.get_yaxis().set_tick_params(direction='out')
ax10.set_ylabel("log scale base 2", labelpad=15)
ax10.text(0.97, 0.97, r"b", transform=ax10.transAxes,
          horizontalalignment='right', verticalalignment='top',
          fontweight="bold")

############
# subplot c
```

```
############
ax11 = plt.subplot(gs[1, 1])

x = np.arange(1.0, np.e ** 4, 0.01)
y = x ** (-0.8)
ax11.plot(x, y, color="cyan", lw=3)
ax11.text(2, 1, r"$y=x^{-0.8}$")
ax11.loglog(x, y)
ax11.set_xscale("log", base=np.e)
ax11.set_yscale("log", base=np.e)
xt = np.exp(np.arange(1, 4, 1))
yt = np.pi ** (np.arange(-3, 2, 1))
ax11.set_xticks(xt)
ax11.set_xticklabels(xt, y=0.15)
ax11.set_yticks(yt)


def ticks_e(y, pos):   # base e
    return r'$e^{:.0f}$'.format(np.log(y))


def ticks_pi(y, pos):  # base pi, why not?
    return r'$\pi^{%+.0f}$'%(np.log(y)/np.log(np.pi))


ax11.xaxis.set_major_formatter(FuncFormatter(ticks_e))
ax11.yaxis.set_major_formatter(FuncFormatter(ticks_pi))
ax11.yaxis.tick_right()
ax11.yaxis.set_label_position("right")
ax11.set_ylabel("right side", labelpad=10)
ax11.text(0.97, 0.97, r"c", transform=ax11.transAxes,
          horizontalalignment='right', verticalalignment='top',
          fontweight="bold")

############
# subplot d
############
ax20 = plt.subplot(gs[2, 0])

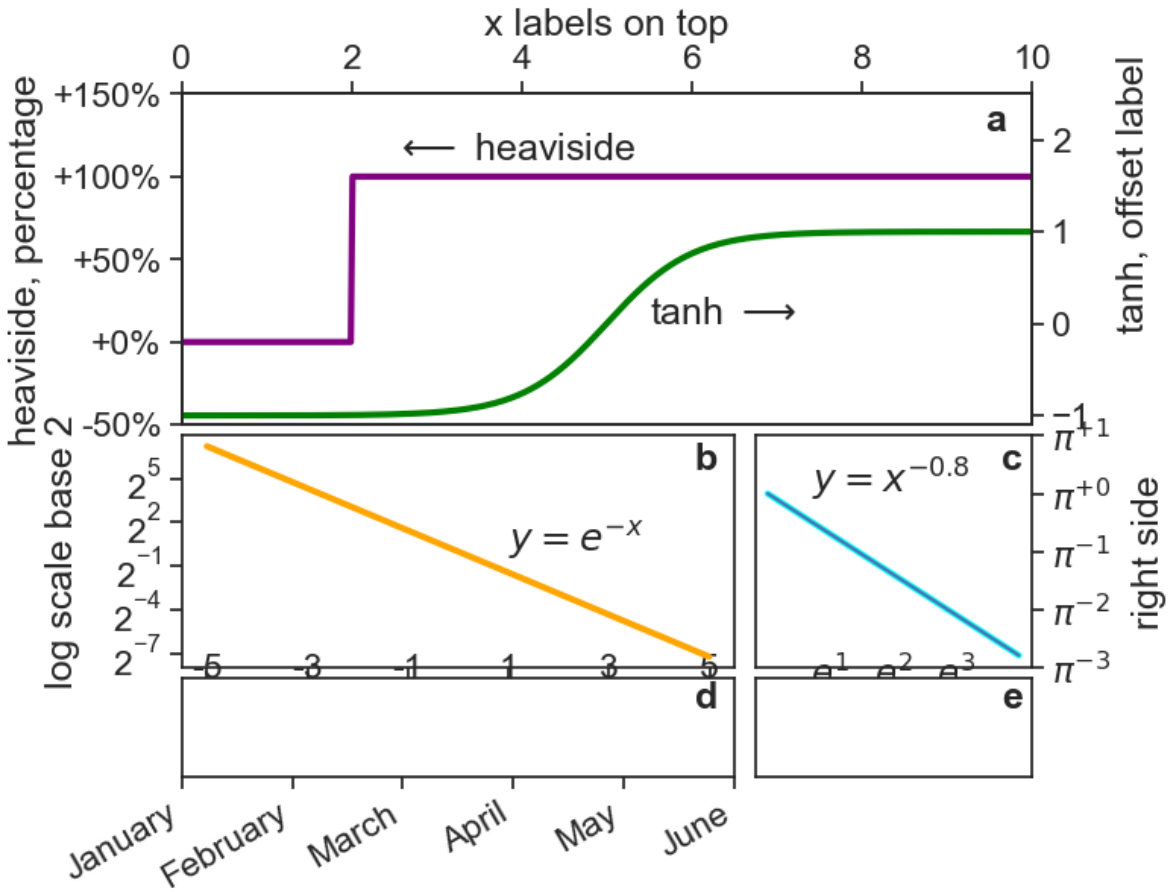
ax20.axis([0, 1, 0, 1])
ax20.set_xticks(np.arange(0, 1.1, 0.2))
ax20.set_xticklabels(["January", "February",
```

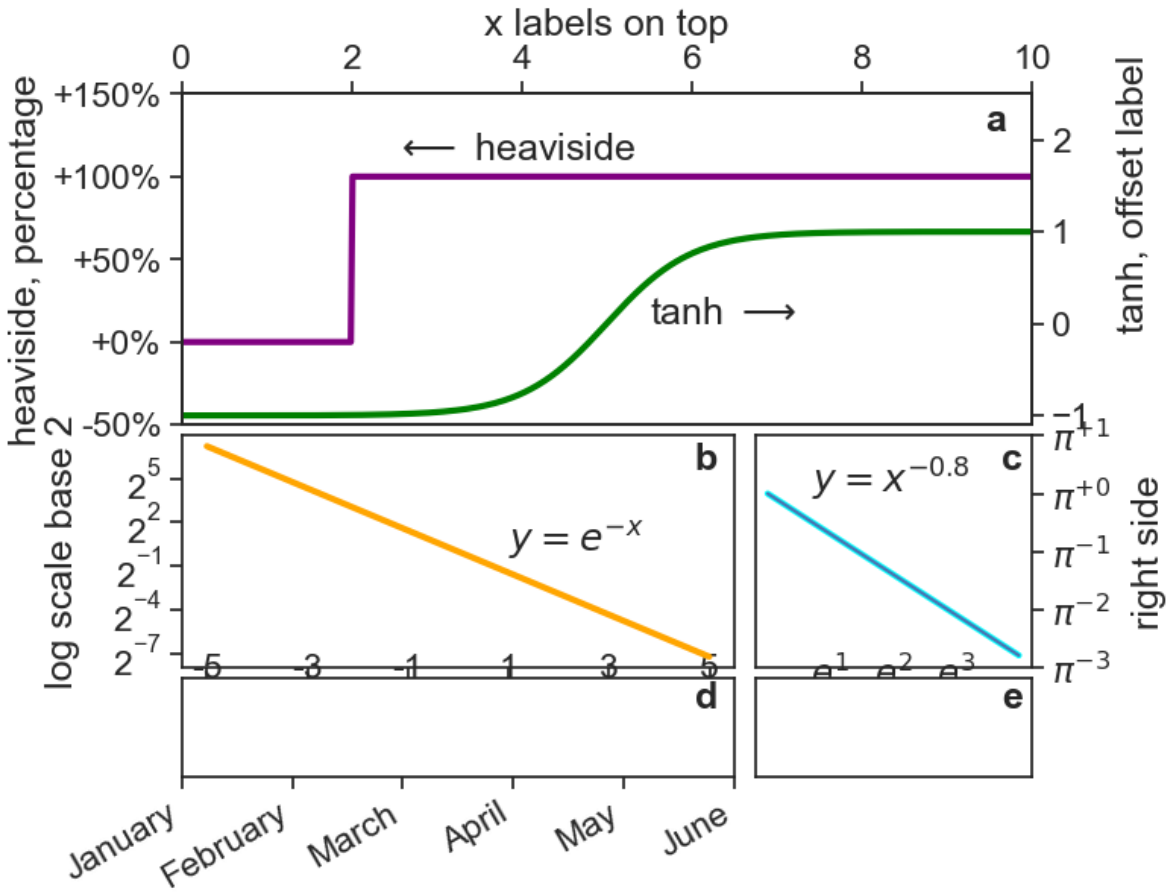
```
                      "March", "April",
                      "May", "June"],
                      rotation=30, horizontalalignment="right")
ax20.set_yticks([])
ax20.text(0.97, 0.97, r"d", transform=ax20.transAxes,
          horizontalalignment='right', verticalalignment='top',
          fontweight="bold")

############
# subplot e
############
ax21 = plt.subplot(gs[2, 1])

ax21.set_xticks([])
ax21.set_yticks([])
ax21.axis([0, 1, 0, 1])
ax21.text(0.97, 0.97, r"e", transform=ax21.transAxes,
          horizontalalignment='right', verticalalignment='top',
          fontweight="bold")

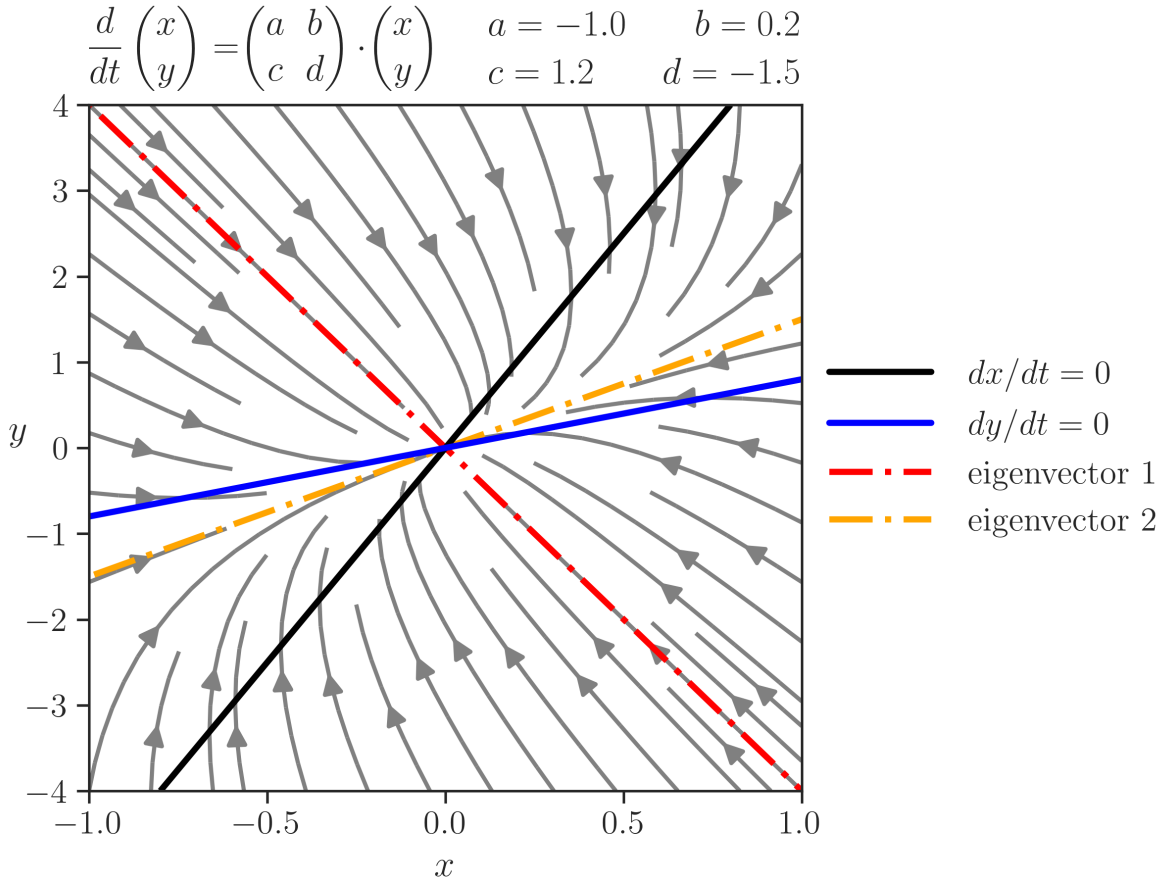
fig.savefig("subplot-grid.png", dpi=300)
fig
```


x labels on top
0
2
4
6
8
10
heaviside, percentage
+150%
+100%
+50%
+0%
-50%
a
← heaviside
tanh →
tanh, offset label
2
1
0
−1
log scale base 2
2^{5}
2^{2}
2^{-1}
2^{-4}
2^{-7}
b
$y = e^{-x}$
-5
-3
-1
1
3
5
c
$y = x^{-0.8}$
e^{1}
e^{2}
e^{3}
right side
π^{+1}
π^{+0}
π^{-1}
π^{-2}
π^{-3}
d
e
January
February
March
April
May
June


x labels on top
0
2
4
6
8
10
heaviside, percentage
+150%
+100%
+50%
+0%
-50%
a
← heaviside
tanh →
tanh, offset label
2
1
0
−1
log scale base 2
2^{5}
2^{2}
2^{-1}
2^{-4}
2^{-7}
b
$y = e^{-x}$
-5
-3
-1
1
3
5
c
$y = x^{-0.8}$
e^{1}
e^{2}
e^{3}
π^{+1}
π^{+0}
π^{-1}
π^{-2}
π^{-3}
right side
d
e
January
February
March
April
May
June

7 Streamplot

Streamplot of a two-dimensional linear system



7.1 Introduction

Streamplot of a two-dimensional linear system, with eigenvectors and nullclines. Python shows LaTeX equations beautifully.

Main features: `meshgrid, streamplot, contour, legend, LaTeX`

7.2 The code

```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
sns.set(style="ticks", font_scale=1.5)
```

make graph look pretty

```
# learn how to configure:
# http://matplotlib.sourceforge.net/users/customizing.html
params = {
          'font.family': 'serif',
          'ps.usedistiller': 'xpdf',
          'text.usetex': True,
          # include here any neede package for latex
          'text.latex.preamble': r'\usepackage{amsmath}',
          }
plt.rcParams.update(params)
```

define parameters, system of equations, and equation for eigenvectors

```
%matplotlib widget
fig, ax = plt.subplots(figsize=(8,6))
fig.subplots_adjust(left=0.08, right=0.68, top=0.87, bottom=0.10,
                    hspace=0.02, wspace=0.02)
# parameters as a dictionary
p = {'a': -1.0, 'b': +0.2,
     'c': +1.2, 'd': -1.5}

# the equations
def system_equations(x,y):
    return [p['a'] * x + p['b'] * y,
            p['c'] * x + p['d'] * y,
           ]
# eigenvectors
```

```
eigen_vec = 100 * np.array([
                           [(p['a'] - p['d'] -
                            np.sqrt((p['a'] - p['d']) ** 2 +
                                    4.0 * p['b'] * p['c'])) /
                            (2.0 * p['c']), 1.0],
                           [(p['a'] - p['d'] +
                            np.sqrt((p['a'] - p['d']) ** 2 +
                                    4.0 * p['b'] * p['c'])) /
                            (2.0 * p['c']), 1.0],
                           ])

min_x, max_x = [-1, 1]
min_y, max_y = [-4, 4]
divJ = 50j
div = 50
# 1st way
# Y, X = np.mgrid[min_y:max_y:div,min_x:max_x:div]
# 2nd way
X, Y = np.meshgrid(np.linspace(min_x, max_x, div),
                   np.linspace(min_y, max_y, div))

# streamplot
density = 2 * [0.80]
minlength = 0.2
arrow_color = 3 * [0.5]
ax.streamplot(X, Y, system_equations(X, Y)[0], system_equations(X, Y)[1],
              density=density, color=arrow_color, arrowsize=2,
              linewidth=2, minlength=minlength)

# eigenvectors
eigen_0, = ax.plot([eigen_vec[0, 0],-eigen_vec[0, 0]],
                   [eigen_vec[0, 1],-eigen_vec[0, 1]],
                   color='red', lw=3, ls="--")
eigen_1, = ax.plot([eigen_vec[1, 0],-eigen_vec[1, 0]],
                   [eigen_vec[1, 1],-eigen_vec[1, 1]],
                   color='orange', lw=3, ls="--")
dash = (7, 2, 1, 2)
eigen_0.set_dashes(dash)
eigen_1.set_dashes(dash)

# nullclines
null_0 = ax.contour(X, Y, system_equations(X, Y)[0],
```

```
                    levels=[0], colors='black', linewidths=3)
null_1 = ax.contour(X, Y,system_equations(X, Y)[1],
                    levels=[0], colors='blue', linewidths=3)
n0, = ax.plot([100,101], [100,101], color='black', linewidth=3)
n1, = ax.plot([100,101], [100,101], color='blue', linewidth=3)

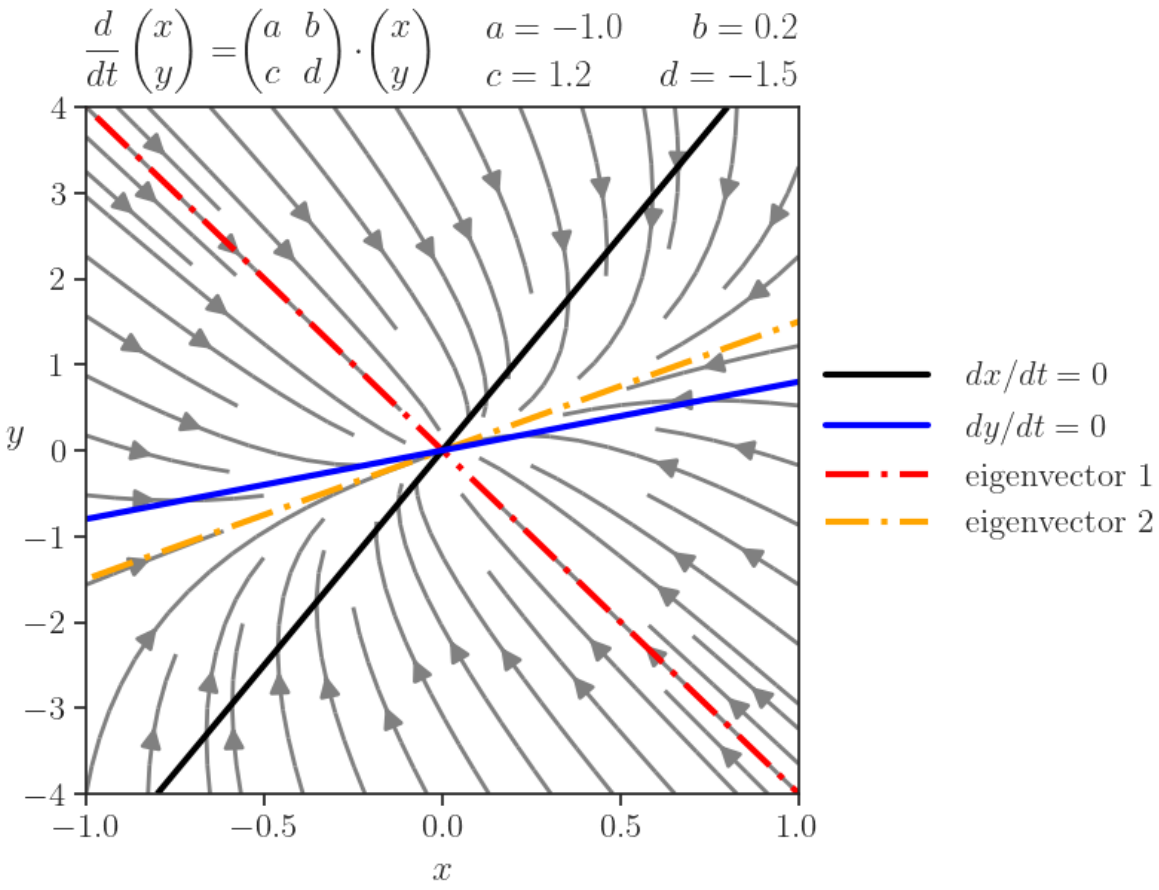
# some text
ax.text(0.0, 1.02, (r"$\displaystyle\frac{d}{dt}\begin{pmatrix}x\\y\end{pmatrix}=$"
                    r"$\begin{pmatrix}a&b\\c&d\end{pmatrix}\cdot$"
                    r"$\begin{pmatrix}x\\y\end{pmatrix}$"),
                    transform=ax.transAxes, va="bottom")
ax.text(1.0, 1.1, r"$a={:.1f}\qquad b={:.1f}$".format(p['a'], p['b']),
        transform=ax.transAxes, ha="right")
ax.text(1.0, 1.03, r"$c={:.1f}\qquad d={:.1f}$".format(p['c'], p['d']),
        transform=ax.transAxes, ha="right")

ax.legend([n0, n1, eigen_0, eigen_1],
          [r'$dx/dt=0$', r'$dy/dt=0$',
           r"eigenvector 1", r"eigenvector 2"],
          loc="center left",
          bbox_to_anchor=(1.0,0.5),
          frameon=False, fancybox=False, shadow=False, ncol=1,
          borderpad=0.5, labelspacing=0.5, handlelength=3, handletextpad=1.1,
          borderaxespad=0.3, columnspacing=2)

ax.set_ylabel(r"$y$", rotation='horizontal')
ax.set_xlabel(r"$x$", labelpad=5)

ax.axis([min_x, max_x, min_y, max_y])

fig.savefig("streamplot.png", dpi=300)
```


$\frac{d}{dt}\begin{pmatrix}x\\y\end{pmatrix} = \begin{pmatrix}a & b\\c & d\end{pmatrix}\cdot\begin{pmatrix}x\\y\end{pmatrix}$
$a = -1.0$
$b = 0.2$
$c = 1.2$
$d = -1.5$
$dx/dt = 0$
$dy/dt = 0$
eigenvector 1
eigenvector 2
x
y

8 Palette

```
import numpy as np
import matplotlib.pyplot as plt
import colorsys
import matplotlib.colors as mcolors
```

```
def get_hls(color):
    """
    Returns the HLS values of a given color.

    Args:
        color (str or tuple): A color name (e.g., 'xkcd:hot pink', 'tab:orange'), a hex code

    Returns:
        tuple: A tuple containing the HLS values (hue, lightness, saturation) of the given co
    """
    try:
        # Convert the input color to RGB
        if isinstance(color, str):
            # If the input is a color name, convert it to RGB
            rgb = mcolors.to_rgb(color)
        elif isinstance(color, tuple) and len(color) == 3:
            # If the input is an RGB triplet, use it directly
            rgb = color
        else:
            # If the input is a hex code, convert it to RGB
            rgb = mcolors.to_rgb(color)

        # Convert RGB to HSL
        r, g, b = rgb
        hls = colorsys.rgb_to_hls(r, g, b)

        return hls

    except ValueError as e:
```

```
        print(f"Error: {e}")
        return None

def get_hls2(color):
    """
    Returns the HLS values of a given color.

    Args:
        color (str or tuple): A color name (e.g., 'xkcd:hot pink', 'tab:orange'), a hex code

    Returns:
        tuple: A tuple containing the HLS values (hue, lightness, saturation) of the given co
    """
    try:
        # Convert the input color to RGB
        if isinstance(color, str):
            # If the input is a color name, convert it to RGB
            rgb = mcolors.to_rgb(color)
            rgb = [int(val * 255) for val in rgb]
        elif isinstance(color, tuple) and len(color) == 3:
            # If the input is an RGB triplet, use it directly
            rgb = color
        else:
            # If the input is a hex code, convert it to RGB
            rgb = mcolors.to_rgb(color)
            rgb = [int(val * 255) for val in rgb]

        # Convert RGB to HSL
        r, g, b = rgb
        hls = colorsys.rgb_to_hls(r / 255, g / 255, b / 255)

        return hls

    except ValueError as e:
        print(f"Error: {e}")
        return None

def new_range(l):
    r = np.arange(-1.0, 1.1, 0.1) + l
    return r[(r<=1.0) & (r>=0.0)]

def rgb_to_hex(rgb):
```

```
    return '#{:02x}{:02x}{:02x}'.format(int(rgb[0]*255), int(rgb[1]*255), int(rgb[2]*255))

def plot_panel(c, type, ax):
    """
    type=0 : hue
    type=1 : lightness
    type=2 : saturation
    """
    r = new_range(c[type])
    if type == 0:
        a_rgb = [colorsys.hls_to_rgb( *((ri, c[1], c[2]) ) ) for ri in r]
        type_name = "hue"
    if type == 1:
        a_rgb = [colorsys.hls_to_rgb( *((c[0], ri, c[2]) ) ) for ri in r]
        type_name = "lightness"
    if type == 2:
        a_rgb = [colorsys.hls_to_rgb( *((c[0], c[1], ri) ) ) for ri in r]
        type_name = "saturation"
    a_hex = [rgb_to_hex(i) for i in a_rgb]
    ax[type].imshow([a_rgb], interpolation='none')
    # Minor ticks
    ax[type].set_xticks(np.arange(-.5, len(a_rgb), 1), minor=True)
    # Gridlines based on minor ticks
    ax[type].grid(which='minor', color='w', linestyle='-', linewidth=2)
    # Remove minor ticks
    ax[type].tick_params(which='minor', bottom=False, left=False)

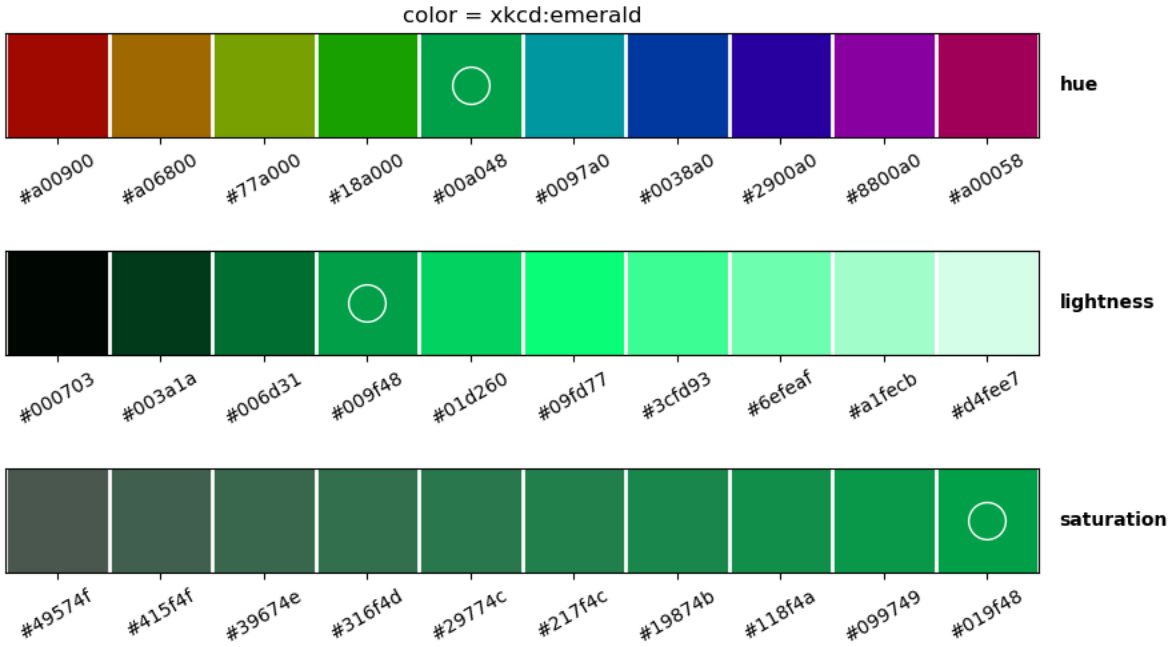
    ax[type].set_xticks(np.arange(len(a_rgb)))
    ax[type].set_xticklabels(a_hex, rotation=30);
    ax[type].set_yticks([])
    ax[type].text(1.02, 0.5, type_name, transform=ax[type].transAxes,
         horizontalalignment='left', verticalalignment='center',
         fontweight="bold")
    mask = np.abs(r-c[type])< 1.0e-8
    idx = np.argwhere(mask)
    ax[type].plot(idx[0], [0], marker='o', markersize=20, markerfacecolor='None', markeredge
    return a_hex

def pallete(color_name):
    fig, ax = plt.subplots(3, 1, figsize=(10,6))
    c = get_hls(color_name)
    hex_codes = plot_panel(c, 0, ax)
```

```
    print("hue array:\n", hex_codes)
    hex_codes = plot_panel(c, 1, ax)
    print("lightness array:\n", hex_codes)
    hex_codes = plot_panel(c, 2, ax)
    print("saturation array:\n", hex_codes)
    ax[0].set_title("color = " + color_name)
    return fig
```

```
pallete('xkcd:emerald');
```

```
hue array:
 ['#a00900', '#a06800', '#77a000', '#18a000', '#00a048', '#0097a0', '#0038a0', '#2900a0', '#8
lightness array:
 ['#000703', '#003a1a', '#006d31', '#009f48', '#01d260', '#09fd77', '#3cfd93', '#6efeaf', '#a
saturation array:
 ['#49574f', '#415f4f', '#39674e', '#316f4d', '#29774c', '#217f4c', '#19874b', '#118f4a', '#0
```




```
fig = pallete('gold')
fig.savefig('palette-gold.png')
```

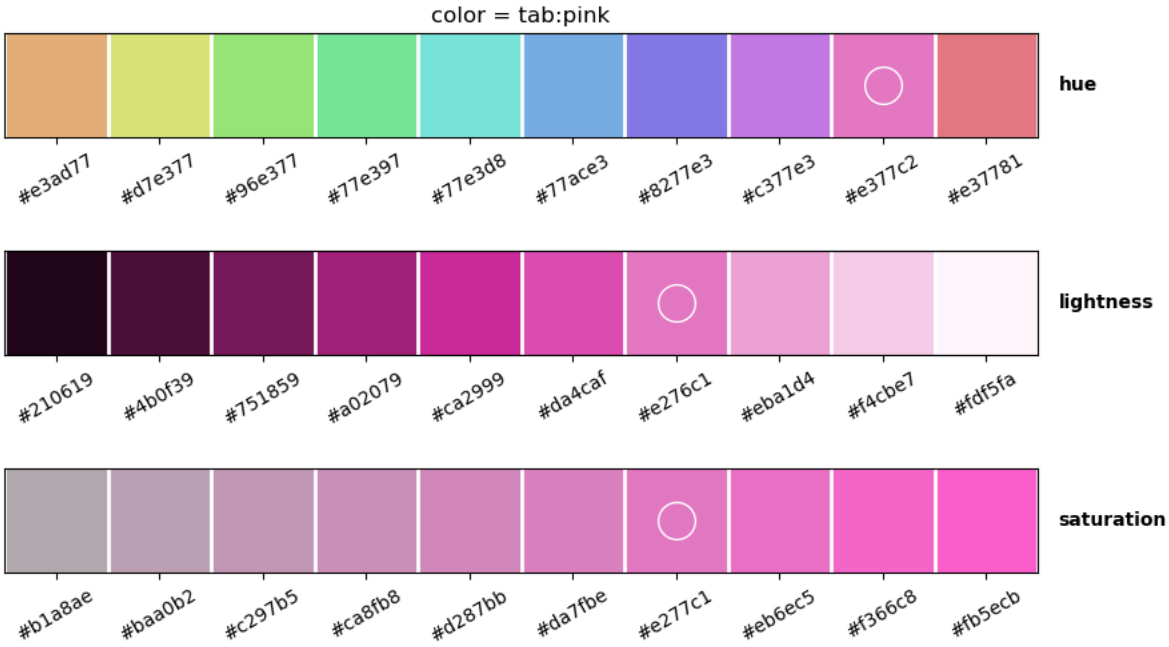
```
hue array:
```

```
 ['#ff3d00', '#ffd600', '#8eff00', '#00ff0a', '#00ffa3', '#00c1ff', '#0028ff', '#7000ff', '#
lightness array:
 ['#322a00', '#655500', '#988000', '#cbab00', '#fed600', '#ffde32', '#ffe665', '#ffef98', '#
saturation array:
 ['#7f7f7f', '#8c8872', '#999166', '#a59959', '#b2a24c', '#bfab3f', '#cbb333', '#d8bc26', '#
```




```
pallete('tab:pink');
```

```
hue array:
 ['#e3ad77', '#d7e377', '#96e377', '#77e397', '#77e3d8', '#77ace3', '#8277e3', '#c377e3', '#
lightness array:
 ['#210619', '#4b0f39', '#751859', '#a02079', '#ca2999', '#da4caf', '#e276c1', '#eba1d4', '#
saturation array:
 ['#b1a8ae', '#baa0b2', '#c297b5', '#ca8fb8', '#d287bb', '#da7fbe', '#e277c1', '#eb6ec5', '#
```


color = tab:pink
hue
#e3ad77
#d7e377
#96e377
#77e397
#77e3d8
#77ace3
#8277e3
#c377e3
#e377c2
#e37781
lightness
#210619
#4b0f39
#751859
#a02079
#ca2999
#da4caf
#e276c1
#eba1d4
#f4cbe7
#fdf5fa
saturation
#b1a8ae
#baa0b2
#c297b5
#ca8fb8
#d287bb
#da7fbe
#e277c1
#eb6ec5
#f366c8
#fb5ecb

9 The time-dependent Ginzburg-Landau equation

Finite differences method applied to a beautiful equation

9.1 Introduction

Simulation of the Time-Dependent Ginzburg-Landau Equation

$$\frac{\mathrm{d}u}{\mathrm{d}t} = u - u^3 + D\nabla^2 u,$$

in 1 and 2 spatial dimensions. This is the simplest example of numerical integration through Finite Differences: - Euler method to advance time - Five-point stencil to compute the laplacian, periodic boundary conditions are assumed. See an example of the output here: https://www.youtube.com/watch?v=JgE9Px7zsQE

9.2 Code

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.axes_grid1 import make_axes_locatable
import time as tm
```

```
# grid size
n = 128
# for 1d simulation write N=(n,)
N=(n,n)
# diffusion coefficient
D = 1.0
# spatial dimensions
L = 100.0
dx = L / n
```

```
x = np.arange(0,L,dx)
# time
t = 0.0
total_time = 3.0
# beware of the Von Neumann stability analysis
# https://en.wikipedia.org/wiki/Von_Neumann_stability_analysis
dt = 0.2 * 0.5 * dx**2 / D
```

define functions

```
def periodic_lap_1d(u,dx=1.0):
    return (+1*np.roll(u,+1)
            +1*np.roll(u,-1)
            -2*u) / dx**2
def periodic_lap_2d(u,dx=1.0):
    return (+1*np.roll(u,+1,axis=0)
            +1*np.roll(u,-1,axis=0)
            +1*np.roll(u,+1,axis=1)
            +1*np.roll(u,-1,axis=1)
            -4*u) / dx**2
f = lambda u: u - u**3
```

initialize and start plotting

```
plt.ion()
fig = plt.figure(1,figsize=(7,6))
plt.clf()
ax = fig.add_subplot(111)
# random initial condition
u = 2*np.random.random(N)-1.0
if len(N) == 1:
    lap = periodic_lap_1d
    p, = ax.plot(x,u)
    ax.axis([x[0],x[-1],-1.1,1.1])
if len(N) == 2:
    lap = periodic_lap_2d
    p = ax.imshow(u,cmap="RdGy", vmin=-1.0, vmax=1.0,extent=[0,L,0,L])
    # create an axes on the right side of ax. The width of cax will be 5%
    # of ax and the padding between cax and ax will be fixed at 0.15 inch.
    divider = make_axes_locatable(ax)
    colorbar_ax = divider.append_axes("right", size="5%", pad=0.15)
```

```
    cbar = fig.colorbar(p, cax=colorbar_ax, ticks=[-1,-0.5,0,0.5,1])
ax.set_title("time={:5.1f}".format(0.0))
```

```
Text(0.5, 1.0, 'time=  0.0')
```

start simulation

```
while t<total_time:
    t += dt
    u = u + dt * (f(u) + D * lap(u,dx) )
    # we don't need to plot again, just to update the data of the plot
    if len(N) == 1:
        p.set_data(x,u)
    if len(N) == 2:
        p.set_data(u)
    ax.set_title("time={:5.1f}".format(t))
    fig.canvas.draw()
```

10 Double Pendulum

The most beautiful chaos

10.1 Introduction

The double pendulum is one of the most famous examples of chaos. Enjoy making your own animations!

10.2 The code

comment the lines below if you want interactive mode, i.e., if you want to see the animation in real time.

```
import matplotlib
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
import numpy as np
import os
from scipy.integrate import ode
```

define equations of motion and other functions

```
def equations(t, y, args):
    """ the equations for the double pendulum """
    x1 = y[0] # x1 = theta1, angle
    x2 = y[1] # x2 = theta2, angle
    p1 = y[2] # p1 = omega1, angular velocity
    p2 = y[3] # p2 = omega2, angular velocity
    l1,l2,m1,m2,g = args
    x1_eq = p1
    x2_eq = p2
```

```
    p1_eq = -((g*(2*m1+m2)*np.sin(x1)+m2*(g*np.sin(x1-2*x2)+2*(l2*p2**2+l1*p1**2*np.cos(x1-x2
    p2_eq = ((l1*(m1+m2)*p1**2+g*(m1+m2)*np.cos(x1)+l2*m2*p2**2*np.cos(x1-x2))*np.sin(x1-x2)
    return [x1_eq, x2_eq, p1_eq, p2_eq]

def calculate_trajectory(args,time,y0):
    """ uses scipy's ode itegrator to simulate the equations """
    t0,t1,dt = time
    r = ode(equations).set_integrator('dopri5')
    r.set_initial_value(y0, t0).set_f_params(args)
    data=[[t0, y0[0], y0[1], y0[2], y0[3] ]]
    while r.successful() and r.t < t1:
        r.integrate(r.t+dt)
        data.append([r.t, r.y[0], r.y[1], r.y[2], r.y[3] ])
    return np.array(data)

def from_angle_to_xy(args,angles):
    """ converts angles into xy positions """
    l1,l2,m1,m2,g = args
    time,theta1,theta2 = angles.T
    x1 =  l1*np.sin(theta1)
    y1 = -l1*np.cos(theta1)
    x2 =  l2*np.sin(theta2) + x1
    y2 = -l2*np.cos(theta2) + y1
    return np.array([time,x1,y1,x2,y2]).T
```

parameters

```
l1 = 0.5 # length of arms
l2 = 0.5
m1 = 1.0 # mass of the pendulum
m2 = 1.0
g  = 10.0 # acceleration of gravity
args = [l1,l2,m1,m2,g]
fps = 80
total_time = 5 # seconds
time = [0.0,total_time,1.0/fps] # start, finish, dt
ic   = [np.pi*0.65, np.pi*1.1, 0.0, 0.0]
```

here the magic happens

```
d = calculate_trajectory(args,time,ic)
data_TXY = from_angle_to_xy(args,d[:,:3])
```

Let's plot stuff, and make a nice movie.
Requrement: ffmpeg

```
make_movie=True
params = {'backend': 'ps',
          'font.size': 20,
          'font.family':'serif',
          'font.serif':['Computer Modern Roman'], # Times, Palatino, New Century Schoolbook,
          'ps.usedistiller': 'xpdf',
          'text.usetex': True,
          }
plt.rcParams.update(params)
plt.ion()
fig = plt.figure(figsize=(9.6,5.4),dpi=100) # 1920x1080
fig.subplots_adjust(left=0, right=1, top=1, bottom=0,hspace=0.02,wspace=0.02)
ax = fig.add_subplot(111)
ax.axis('off') # no frame

def plot_last_seconds(data,index):
    """ Plots a line with the trajectory of the tip of pendulum 2 (x2,y2)
    """
    how_long = 1.0 # seconds
    n = int(how_long/time[2])
    to_plot = data[:index,:]
    if index < n:
        prepend =  np.tile(data[0],(n-index,1))
        to_plot = np.vstack([prepend,to_plot])
        index = n
    colormap = plt.cm.Greys_r
    colors = [colormap(i) for i in np.linspace(0.0, 1.0, n-1)]
    plots = []
    for j in np.arange(n-1):
        p, = ax.plot(to_plot[index-j-1:index-j+1,3],to_plot[index-j-1:index-j+1,4],
                color=colors[j], zorder=-1)
        plots.append(p)
    return plots

# "plot" returns a tuple of line objects, thus the comma
t,x1,y1,x2,y2 = data_TXY[0]
```

```
line1, = ax.plot([0.0,x1], [0.0,y1], 'r-')
line2, = ax.plot([x1,x2], [y1,y2], 'r-')
circ1, = ax.plot([x1], [y1], 'ro',markersize=10)
circ2, = ax.plot([x2], [y2], 'ro',markersize=10)
sizeY = 1.2
ax.axis([-sizeY*16/9,sizeY*16/9,-sizeY,sizeY])

frame_names = []
tex=ax.text(0.0,0.85,'',ha="center")

for i,v in enumerate(data_TXY):
    t,x1,y1,x2,y2 = v
    # print("t={:.2f}".format(t)) # you might want to know how things are going...
    line1.set_data([0.0,x1],[0.0,y1])
    line2.set_data([x1,x2],[y1,y2])
    circ1.set_data([x1],[y1])
    circ2.set_data([x2],[y2])

    # plot_last_seconds considerably slows down the simulation,
    # but makes it much prettier...
    pls = plot_last_seconds(data_TXY,i+1)
    tex.set_text(r"$t={:.3f}$ s".format(t))
    fig.canvas.draw()
    if make_movie:
        fname = "_tmp{:05d}.png".format(i)
        frame_names.append(fname)
        fig.savefig(fname,bbox_inches='tight')
    for k in pls:
        k.remove()

if make_movie:
    frames = "_tmp%5d.png"
    frames = "_tmp%5d.png"
    movie_command = "ffmpeg -y -r {:} -i {:} double.mp4".format(fps,frames)
    os.system(movie_command)

    for fname in frame_names:
        os.remove(fname)
```

11 Hilbert Curve

It will fill the plane!

11.1 Introduction

The Hilbert curve is calculated iteratively. Check out this: Hilbert_curve: Representation_as_Lindenmayer_system

11.2 The code

```
# comment this to enable interactive mode
import matplotlib
matplotlib.use('AGG')
```

```
import numpy as np
import matplotlib.pyplot as plt
import os
```

```
def apply_rules(s):
    """ Hilbert Curve as a Lindenmayer system (L-system)
    https://en.wikipedia.org/wiki/Hilbert_curve#Representation_as_Lindenmayer_system"""
    s=s.replace("a","-Bf+AfA+fB-")  # capital letters "A" and "B" so that the second operati
    s=s.replace("b","+Af-BfB-fA+")  # doesn't apply to the changes already made
    return s.lower() # make everyone lowercase

axiom = "a"
n=3 # number of iterations
# displacements, ordered in a counter-clockwise direction
dxdy = np.array([[ 1, 0],    # right
                 [ 0, 1],    # up
                 [-1, 0],    # left
```

```
                  [ 0,-1] ])  # down
# displacement is of size 1, so the higher n is, the greater the domain
length = 2**n-1; margin = 0.05*length
domain = [0-margin,length+margin,0-margin,length+margin] # a 5% margin around the curve
s = axiom # string to iterate upon

for i in np.arange(n):
    s = apply_rules(s)
```

```
make_movie=True
plt.ion() # interactive mode disabled if "matplotlib.use('AGG')"
fig = plt.figure(figsize=(6,6))
ax = fig.add_subplot(111)
ax.axis('off')                          # no frame
ax.axis(domain)                         # domain size
ax.set_aspect('equal')                  # square look
ax.set_xticks([]); ax.set_yticks([])  # no ticks
ax.set_title(r"$n = {:d}$".format(n))
plt.show()

# "a" and "b" can be erased now
s=s.replace("a","")
s=s.replace("b","")

frame_names = []  # these two are only relevant if make_movie==True
frame_counter=0

p = np.array([[0.0,0.0]]) # this is the starting point (0,0)
p_plot, = plt.plot(p[:,0],p[:,1],color="black")

# iterate on the string s
for i,c in enumerate(s):
    # uncomment to see how fast things are going
    # print("{:d}/{:d}".format(i,len(s)))

    # rotations "+" and "-" change the displacement array dxdy
    # "+" means clockwise rotation
    if c == '+': dxdy = np.roll(dxdy,+1,axis=0)
    # "-" means counter-clockwise rotation
    if c == '-': dxdy = np.roll(dxdy,-1,axis=0)
    # forward "f"
    if c == 'f':
```

```
        # add one more point to array p
        p = np.vstack([p, [p[-1,0]+dxdy[0,0],p[-1,1]+dxdy[0,1]] ])
        # update p_plot data, this is MUCH faster that plotting
        # several line segments separately
        p_plot.set_data(p[:,0],p[:,1])
        fig.canvas.draw()
        if make_movie:
            fname = "_tmp{:05d}.png".format(frame_counter)
            frame_names.append(fname)
            fig.savefig(fname,bbox_inches='tight',resolution=300)
        frame_counter += 1
```

```
if make_movie:
    frames = "_tmp%5d.png"
    # movie_command = "mencoder mf://*.png -mf fps=24:type=png --ovc lavc -lavcopts vcodec=mp

    # we might have other .png figures in the directory
    # in this case, use the code below
    f = open("png_list.txt", "w")
    for i in frame_names:
        f.write(i+"\n")
    f.close()
    movie_command = "mencoder mf://@png_list.txt -mf fps=24:type=png -ovc lavc -lavcopts vcod

    err=os.system(movie_command)
    if err!=0:
        raise RuntimeError("Couldn't run mencoder.  Data in tmp*.png files")
    for fname in frame_names:
        os.remove(fname)

    # we now have one video ready.
    # if you want to join several videos, use this:
    # sudo apt-get install gpac
    # MP4Box -cat part1.avi -cat part2.avi -new joinedfile.avi
```

12 A hysteresis mechanism

The way up is not the same as the way down

12.1 Introduction

Hysteresis mechanism created by bistability of states.

Energy function:

$$f = u^4 - 2u^2 + hu$$

12.2 The code

```
# comment these lines if you want interactive mode,
# i.e., if you want to see the animation in real time.
import matplotlib
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
import numpy as np
import os
import sympy
from scipy.integrate import ode

# learn how to configure: http://matplotlib.sourceforge.net/users/customizing.html
params = {#'backend': 'GTKAgg',
          'legend.handlelength': 2.5,
          'legend.borderaxespad': 0,
          'font.family':'serif',
          'font.size': 18,
          'font.serif':['Times'], # Times, Palatino, New Century Schoolbook, Bookman, Compute
          'ps.usedistiller': 'xpdf',
          'text.usetex': True,
```

```
          }

plt.rcParams.update(params)
fig=plt.figure(1,figsize=(9.6,5.4),dpi=100) # 1920x1080 # figsize accepts only inches. if you
fig.clf()
fig.subplots_adjust(left=0.07, right=0.93,top=0.90, bottom=0.12,hspace=0.02,wspace=0.10)

Hlim=2.5 # parameter range from -Hlim to Hlim

ax1=fig.add_subplot(121)
ax1.set_xticks([])
ax1.set_yticks([])
ax1.set_xlabel(r'System response',labelpad=12)
ax1.set_ylabel('Energy',labelpad=12)
ax1.axis([-Hlim,Hlim,-5,5])

ax2=fig.add_subplot(122)
ax2.set_xticks([])
ax2.set_yticks([])
ax2.set_xlabel(r'Parameter',labelpad=12)
ax2.set_ylabel(r'System response',labelpad=12)
ax2.yaxis.set_label_position("right")
ax2.axis([-Hlim*1.2,Hlim*1.2,-2,2])

frame_names = []
frame_index = 0
make_movie=True
plt.ion()
```

```
# energy function and its derivative
f = lambda u,h: u**4-2*u**2+h*u
fprime = lambda u,h: sympy.diff(f(u,h),u)

Hinit=Hlim
ulim=2.5 # system response axis, from -ulim to ulim
u = np.linspace(-ulim,ulim,101)

x = sympy.Symbol('x')

def res(h):
    """System response is one of the real roots
       of the energy function derivative
```

```
    """
    # derivative roots, complex
    resp = sympy.solvers.solve(fprime(x,h),x)
    # numerical evaluation
    resp = map(sympy.N,resp)
    # let's check which roots are real
    isreal = len(resp)*[False]
    for i in range(len(resp)):
        # negligible imaginary component
        if np.abs(sympy.functions.im(resp[i]))<1e-15:
            resp[i]=sympy.functions.re(resp[i])
            isreal[i]=True
    resp = np.array(resp)
    # return only real roots
    return resp[np.array(isreal)]

# let's plot stuff, and make a nice movie
#### left plot, ax1 ####
line_func, = ax1.plot(u,f(u,Hinit),lw=2,color='black')
# ball color
ball_color = "blue"
# minimum = the smallest root, the leftmost root
mini = np.min(res(Hinit)) # calculated for initial parameter value
boost = 0.22 # so that ball sits on top of the curve
# plot ball
ball_u, = ax1.plot([mini],[f(mini,Hinit)+boost],'o',
                markersize=12, markerfacecolor=ball_color)

#### right plot, ax2 ####
# build empty hysteresis array, we will add values
# as simulation progresses
deetype = np.dtype([('h', 'float64'), ('u', 'float64')])
hysteresis = np.array([(Hinit,mini)],dtype=deetype)
line_hyst, = ax2.plot(hysteresis['h'],hysteresis['u'], lw=2,color='black')
ballH, = ax2.plot([hysteresis['h'][-1]],[hysteresis['u'][-1]],'o',
                 markersize=12, markerfacecolor=ball_color)
plt.show()
```

```
# time to simulate
Total_time = 15 # seconds
fps = 24 # frames per second
```

```
# divided by 2 because we ramp down then up
param_vec = np.linspace(Hlim,-Hlim,Total_time*fps/2)

# ramp down
for H in param_vec:
    line_func.set_data(u,f(u,H)) # update line on the left
    mini = np.min(res(H)) # calculate new minimum
    ball_u.set_data([mini],[f(mini,H)+boost]) # update ball on the left
    new_line = np.array([(H,mini)],dtype=deetype) # create new line
    # append new line to hysteresis array
    hysteresis = np.concatenate((hysteresis,new_line))
    line_hyst.set_data(hysteresis['h'],hysteresis['u']) # update line
    ballH.set_data([hysteresis['h'][-1]],[hysteresis['u'][-1]]) # update ball on the right
    fig.canvas.draw()
    if make_movie:
        fname = "_tmp{:05d}.png".format(frame_index)
        frame_names.append(fname)
        fig.savefig(fname,dpi=200)
        frame_index+=1

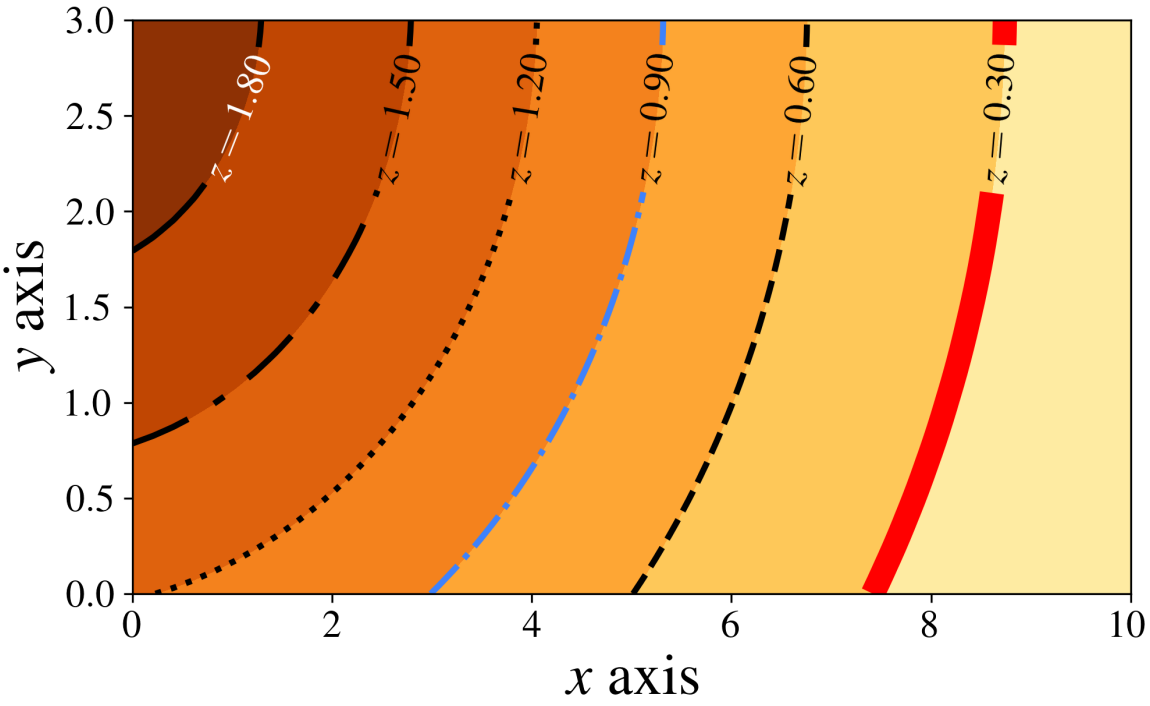
# ramp up
for H in param_vec[::-1]: # just reverse parameter array
    line_func.set_data(u,f(u,H))
    maxi = np.max(res(H)) # everything is the same, but now with maximum
    ball_u.set_data([maxi],[f(maxi,H)+boost])
    new_line = np.array([(H,maxi)],dtype=deetype)
    hysteresis = np.concatenate((hysteresis,new_line))
    line_hyst.set_data(hysteresis['h'],hysteresis['u'])
    ballH.set_data([hysteresis['h'][-1]],[hysteresis['u'][-1]])
    fig.canvas.draw()
    if make_movie:
        fname = "_tmp{:05d}.png".format(frame_index)
        frame_names.append(fname)
        fig.savefig(fname,dpi=200)
        frame_index+=1

if make_movie:
    frames = "_tmp%5d.png"
    movie_command = "ffmpeg -y -r {:} -i {:} ball.mp4".format(fps,frames)
    os.system(movie_command)

    for fname in frame_names:
```

```
        # pass
        os.remove(fname)
```

13 Contour plot




```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
# %matplotlib widget
```

```
fig_size = (8, 6)
tick_size = 16
if True:
    # this is a latex constant, don't change it.
    pts_per_inch = 72.27
    # write "\the\textwidth" (or "\showthe\columnwidth" for a 2 collumn text)
    text_width_in_pts = 246.0
    # inside a figure environment in latex, the result will be on the
```

```
    # dvi/pdf next to the figure. See url above.
    text_width_in_inches = text_width_in_pts / pts_per_inch
    # make rectangles with a nice proportion
    golden_ratio = 0.618
    # figure.png or figure.eps will be intentionally larger, because it is prettier
    inverse_latex_scale = 2
    # when compiling latex code, use
    # \includegraphics[scale=(1/inverse_latex_scale)]{figure}
    # we want the figure to occupy 2/3 (for example) of the text width
    fig_proportion = (3.0 / 3.0)
    csize = inverse_latex_scale * fig_proportion * text_width_in_inches
    # always 1.0 on the first argument
    fig_size = (1.0 * csize,golden_ratio * csize)
    # find out the fontsize of your latex text, and put it here
    text_size = inverse_latex_scale * 12
    tick_size = inverse_latex_scale * 8
    # learn how to configure:
    # http://matplotlib.sourceforge.net/users/customizing.html
    params = {'backend': 'ps',
            'axes.labelsize': text_size,
            'legend.fontsize': tick_size,
            'legend.handlelength': 2.5,
            'legend.borderaxespad': 0,
            'xtick.labelsize': tick_size,
            'ytick.labelsize': tick_size,
            'font.family': 'serif',
            'font.size': text_size,
            # Times, Palatino, New Century Schoolbook,
            # Bookman, Computer Modern Roman
            'font.serif': ['Times'],
            'ps.usedistiller': 'xpdf',
            'text.usetex': True,
            'figure.figsize': fig_size,
            # include here any neede package for latex
            'text.latex.preamble': r'\usepackage{amsmath}',
            }
    plt.rcParams.update(params)
```

```
fig, ax = plt.subplots(figsize=fig_size)
fig.subplots_adjust(left=0.12, right=0.96, top=0.96, bottom=0.18,
                    hspace=0.02, wspace=0.02)
```

```
def truncate_colormap(cmap, minval=0.0, maxval=1.0, n=256):
    new_cmap = matplotlib.colors.LinearSegmentedColormap.from_list(
        'trunc({n},{a:.2f},{b:.2f})'.format(n=cmap.name, a=minval, b=maxval),
        cmap(np.linspace(minval, maxval, n)))
    return new_cmap
cmap = plt.get_cmap('YlOrBr')
my_cmap = truncate_colormap(cmap, 0.2, 0.9)

minX = 0
maxX = 10
minY = 0
maxY = 3
N = 50j
y, x = np.mgrid[minY:maxY:N, minX:maxX:N]
z = 2 * np.exp(-(0.02 * (x + 1) ** 2 + 0.05 * (y - 3.1) **2 ))

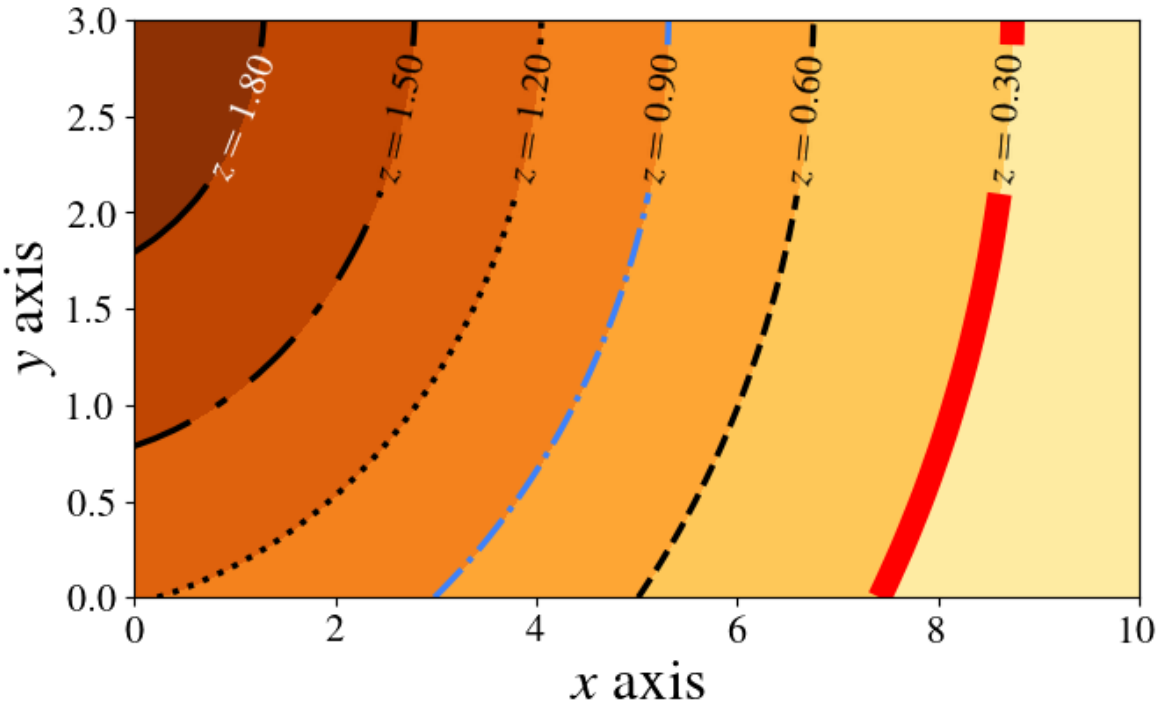
divisions = np.arange(0.3, 2.1, 0.3)
divisions2 = np.append(divisions, 2.5)
divisions2 = np.append(-0.5, divisions2)
# contour filled with colors
ax.contourf(x, y, z, divisions2, cmap=my_cmap, vmin=0.0,vmax=2.0)


line_styles = ['solid', 'dashed', 'dashdot', 'dotted', (0, (10,5,2,5)), 'solid', ]
line_widths = [10, 2.5, 2.5, 2.5, 2.5, 2.5]
colors = ['red', 'black', 'xkcd:dodger blue', 'black', 'black', 'black']

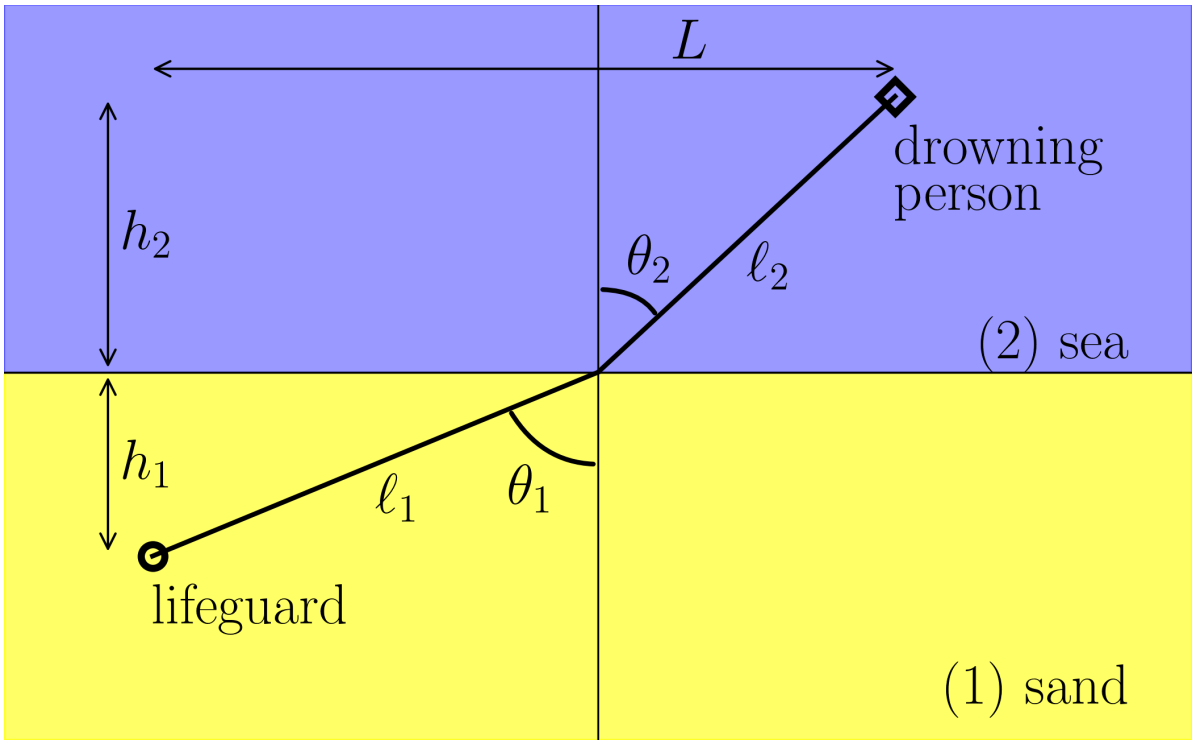
# contour lines
cont = ax.contour(x, y, z, divisions,
                  colors=colors,
                  linewidths=line_widths,
                  linestyles=line_styles)
# labels
manual_locations = [(1.0, 2.5), (2.5, 2.5),
                    (3.8, 2.5), (5.0, 2.5),
                    (6.2, 2.5), (8.3, 2.5)]
ax.clabel(cont, inline=1, fontsize=tick_size,
          fmt=r'$z=%.2f$', manual=manual_locations,
          colors=5 * ['black'] + ['white'])

ax.set_xlabel(r"$x$ axis")
ax.set_ylabel(r"$y$ axis")
```

```
fig.savefig('contour_plot.png', dpi=300)
```



14 Best path




```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
# %matplotlib widget
```

```
# this is a latex constant, don't change it.
pts_per_inch = 72.27
# write "\the\textwidth" (or "\showthe\columnwidth" for a 2 collumn text)
text_width_in_pts = 252.0
# inside a figure environment in latex, the result will be on the
# dvi/pdf next to the figure. See url above.
text_width_in_inches = text_width_in_pts / pts_per_inch
# make rectangles with a nice proportion
```

```
golden_ratio = 0.618
# figure.png or figure.eps will be intentionally larger, because it is prettier
inverse_latex_scale = 2
# when compiling latex code, use
# \includegraphics[scale=(1/inverse_latex_scale)]{figure}
# we want the figure to occupy 2/3 (for example) of the text width
fig_proportion = (3.0 / 3.0)
csize = inverse_latex_scale * fig_proportion * text_width_in_inches
# always 1.0 on the first argument
fig_size = (1.0 * csize,golden_ratio * csize)
# find out the fontsize of your latex text, and put it here
text_size = inverse_latex_scale * 12
tick_size = inverse_latex_scale * 8
# learn how to configure:
# http://matplotlib.sourceforge.net/users/customizing.html
params = {'axes.labelsize': text_size,
          'legend.fontsize': tick_size,
          'legend.handlelength': 2.5,
          'legend.borderaxespad': 0,
          'xtick.labelsize': tick_size,
          'ytick.labelsize': tick_size,
          'font.family': 'serif',
          'font.size': text_size,
          'font.serif': ['Computer Modern Roman'],
          'text.usetex': True,
          'figure.figsize': fig_size,
          'text.latex.preamble': r'\usepackage{amsmath}',
          }
plt.rcParams.update(params)
fig, ax = plt.subplots(figsize=fig_size)
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0.02, wspace=0.02)

origin = [0, 0]
lifeguard = [-3, -2]
drowning = [2, 3]
xlim = [-4, 4]
ylim = [-4, 4]

##### drowning #####
# line
ax.plot([origin[0], drowning[0]], [origin[1], drowning[1]],
```

```
        color="black", lw=2)
# diamond
ax.plot(drowning[0], drowning[1], "D", markerfacecolor="None", markersize=10, markeredgecolo
        markeredgewidth=3, color="black")
# explanation
ax.text(drowning[0], drowning[1] - 0.3, r"drowning", verticalalignment="top")
ax.text(drowning[0], drowning[1] - 0.8, r"person", verticalalignment="top")

##### lifeguard #####
# line
ax.plot([origin[0], lifeguard[0]], [origin[1], lifeguard[1]],
        color="black", lw=2)
# circle
ax.plot(lifeguard[0], lifeguard[1], "o", markerfacecolor="None", markeredgecolor="black",
        markersize=10, markeredgewidth=3, color="black")
# explanation
ax.text(lifeguard[0], lifeguard[1] - 0.3, r"lifeguard",
        verticalalignment="top")

##### sand and sea background #####
sand = matplotlib.patches.Rectangle([xlim[0], ylim[0]],
                                    (xlim[1] - xlim[0]),
                                    (ylim[1] - ylim[0]) / 2.0,
                                    color="yellow", alpha=0.6)
ax.add_patch(sand)
sea = matplotlib.patches.Rectangle([xlim[0], 0],
                                   (xlim[1] - xlim[0]),
                                   (ylim[1] - ylim[0]) / 2.0,
                                   color="blue", alpha=0.4)
ax.add_patch(sea)
###### sand #####
ax.text(0.95, 0.05, r"(1) sand",
        transform=ax.transAxes, horizontalalignment='right')
###### sea #####
ax.text(0.95, 0.50, r"(2) sea",
    transform=ax.transAxes, horizontalalignment='right',
    verticalalignment="bottom")

ax.spines['left'].set_position('zero')
ax.spines['right'].set_color('none')
ax.spines['bottom'].set_position('zero')
ax.spines['top'].set_color('none')
```

```
ax.set(xlim=xlim,
       ylim=ylim,
       xticks=[],
       yticks=[])

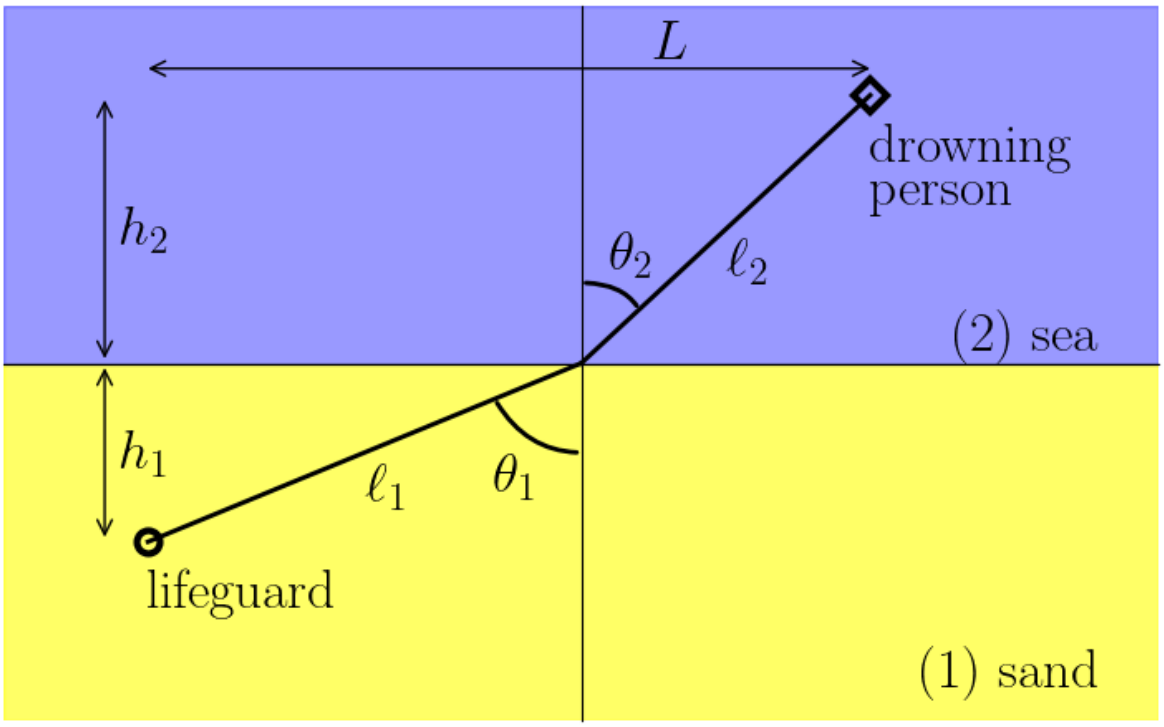
ax.annotate("",
            xy=(lifeguard[0] - 0.3, lifeguard[1]), xycoords='data',
            xytext=(lifeguard[0] - 0.3, 0), textcoords='data',
            size=tick_size,
            arrowprops=dict(arrowstyle="<->",
                            connectionstyle="arc3"),
            )
ax.text(lifeguard[0] - 0.2, lifeguard[1] / 2.0, r"$h_1$",
        verticalalignment="center", horizontalalignment="left")
# h_2
ax.annotate("",
            xy=(lifeguard[0] - 0.3, drowning[1]), xycoords='data',
            xytext=(lifeguard[0] - 0.3, 0), textcoords='data',
            size=tick_size,
            arrowprops=dict(arrowstyle="<->",
                            connectionstyle="arc3"),
            )
ax.text(lifeguard[0] - 0.2, drowning[1] / 2.0, r"$h_2$",
        verticalalignment="center", horizontalalignment="left")
# L
ax.annotate("",
            xy=(lifeguard[0], drowning[1] + 0.3), xycoords='data',
            xytext=(drowning[0], drowning[1] + 0.3), textcoords='data',
            size=tick_size,
            arrowprops=dict(arrowstyle="<->",
                            shrinkA=0, shrinkB=0,
                            connectionstyle="arc3"),
            )
ax.text((lifeguard[1] - lifeguard[0]) / 2.0, drowning[1] + 0.3, r"$L$",
        verticalalignment="bottom", horizontalalignment="left")

# l1
ax.text(lifeguard[0] / 2.0, 1.10 * lifeguard[1] / 2.0, r"$\ell_1$",
        verticalalignment="top", horizontalalignment="left")
# l2
ax.text(drowning[0] / 2.0, 0.95 * drowning[1] / 2.0, r"$\ell_2$",
        verticalalignment="top", horizontalalignment="left")
```

```
# theta_1
ax.annotate("",
            xy=(0, 0.5 * lifeguard[1]), xycoords='data',
            xytext=(0.2 * lifeguard[0], 0.2 * lifeguard[1]), textcoords='data',
            size=tick_size,
            arrowprops=dict(arrowstyle="-", lw=2,
                            connectionstyle="angle3,angleA=-60,angleB=0"),
            )
ax.text(0.1 * lifeguard[0], 0.5 * lifeguard[1], r"$\theta_1$",
        verticalalignment="top", horizontalalignment="right")

# theta_2
ax.annotate("",
            xy=(0, 0.3 * drowning[1]), xycoords='data',
            xytext=(0.2 * drowning[0], 0.2 * drowning[1]), textcoords='data',
            size=tick_size,
            arrowprops=dict(arrowstyle="-", lw=2,
                            connectionstyle="angle3,angleA=120,angleB=0"),
            )
ax.text(0.1 * drowning[0], 0.5 * drowning[1], r"$\theta_2$",
        verticalalignment="top", horizontalalignment="left")

fig.savefig('best_path.png', dpi=300)
```


L
drowning
person
h_2
θ_2
ℓ_2
(2) sea
h_1
ℓ_1
θ_1
lifeguard
(1) sand

15 Logo

```
import matplotlib.pyplot as plt
import matplotlib as mpl
import os
from matplotlib import font_manager
mpl.rcParams['text.usetex'] = True
mpl.rcParams['font.family'] = 'serif'
mpl.rcParams['font.serif'] = ['Times']
mpl.rcParams['text.latex.preamble'] = r'\usepackage{amsmath}'
import numpy as np

# %matplotlib widget
```

15.1 Equation for an offset parabola

```
x0 = 0.25
x = np.linspace(x0, 1, 1001)
def parabola(x):
    return (x - x0)**2 / (1 - x0)**2
y = parabola(x)
```

15.2 The standard logo

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)
```

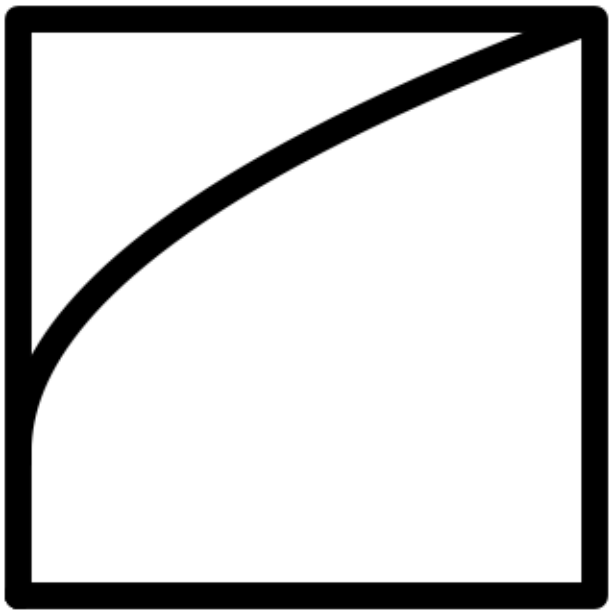
```
for spine in ax.spines.values():
    spine.set_visible(False)

width = 10
solid_capstyle = 'round' # 'butt', 'round', 'projecting'
solid_joinstyle = 'round' # 'miter', 'round', 'bevel'

frame_path_x = np.array([0, 1, 1, 0, 0])
frame_path_y = np.array([0, 0, 1, 1, 0])
ax.plot(frame_path_x, frame_path_y, color="black", lw=width, clip_on=False,
        solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

ax.plot(y, x, color="black", lw=width)
ax.set(xlim=[0,1],
       ylim=[0,1],
       xticks=[],
       yticks=[],)

# fig.savefig("logo_white_bg_withborder.png", dpi=600, transparent=False, bbox_inches=None)
# fig.savefig("logo_transparent_bg_withborder.png", dpi=600, transparent=True, bbox_inches=No
# fig.savefig("logo_white_bg.png", dpi=600, transparent=False, bbox_inches="tight")
fig.savefig("logo_transparent_bg.png", dpi=600, transparent=True, bbox_inches="tight")
# fig.savefig("figure.svg", transparent=True, bbox_inches="tight")
```



15.3 Logo in white

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)

for spine in ax.spines.values():
    spine.set_visible(False)

width = 10
solid_capstyle = 'round' # 'butt', 'round', 'projecting'
solid_joinstyle = 'round' # 'miter', 'round', 'bevel'

frame_path_x = np.array([0, 1, 1, 0, 0])
frame_path_y = np.array([0, 0, 1, 1, 0])
ax.plot(frame_path_x, frame_path_y, color="white", lw=width, clip_on=False,
        solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

ax.plot(y, x, color="white", lw=width)
ax.set(xlim=[0,1],
       ylim=[0,1],
       xticks=[],
       yticks=[],)

fig.savefig("logo_inverted.png", dpi=600, transparent=True, bbox_inches="tight")
```

15.4 Annotated logo

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)

for spine in ax.spines.values():
    spine.set_visible(False)

ax.plot(frame_path_x, frame_path_y, color="black", lw=width, clip_on=False,
        solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

ax.plot(y, x, color="black", lw=width)
ax.set(xlim=[0,1],
       ylim=[0,1],
       xticks=[],
       yticks=[],)
ax.set_xlabel(r"saddle-node bifurcation",
              fontsize=20, color="red", labelpad=10)
```

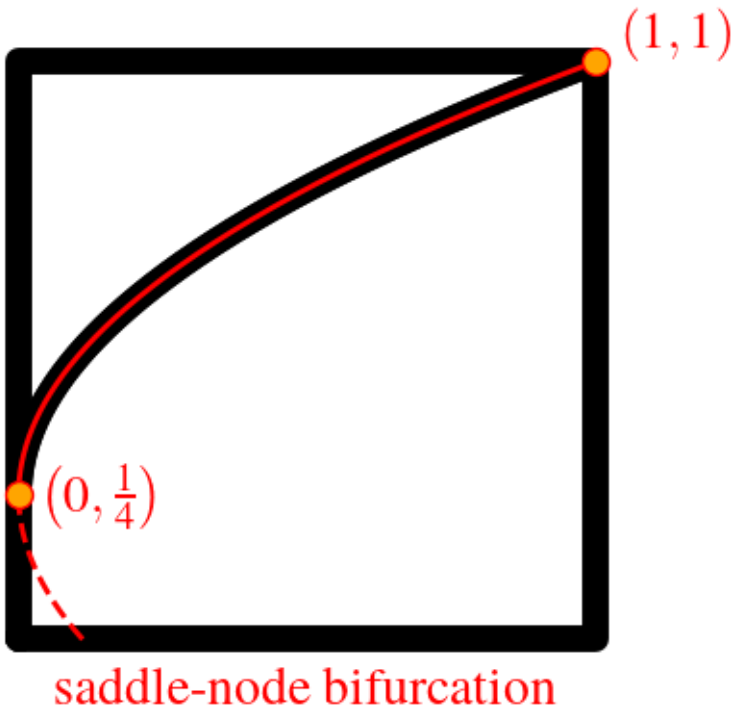
```
x2 = np.linspace(0, 0.25, 1001)
y2 = parabola(x2)
ax.plot(y, x, color="black", lw=10)
ax.plot(y, x, color="red", lw=2, clip_on=False)
ax.plot(y2, x2, color="red", lw=2, clip_on=False, ls="--")

ax.plot(0, 0.25, 'o', markersize=10, clip_on=False, markerfacecolor="orange", markeredgecolo
ax.text(0.05, 0.25, r"$\left(0, \tfrac{1}{4}\right)$", fontsize=20, color="red", va="center"

ax.plot(1, 1, 'o', markersize=10, clip_on=False, markerfacecolor="orange", markeredgecolor="
ax.text(1.05, 1.05, r"$\left(1,1\right)$", fontsize=20, color="red", va="center", ha="left")

# fig.savefig("logo_explanation.png", dpi=600, transparent=False, bbox_inches=None) # bbox_in
```

```
Text(1.05, 1.05, '$\\left(1,1\\right)$')
```



15.5 Chunky logo to be converted into favicon

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)

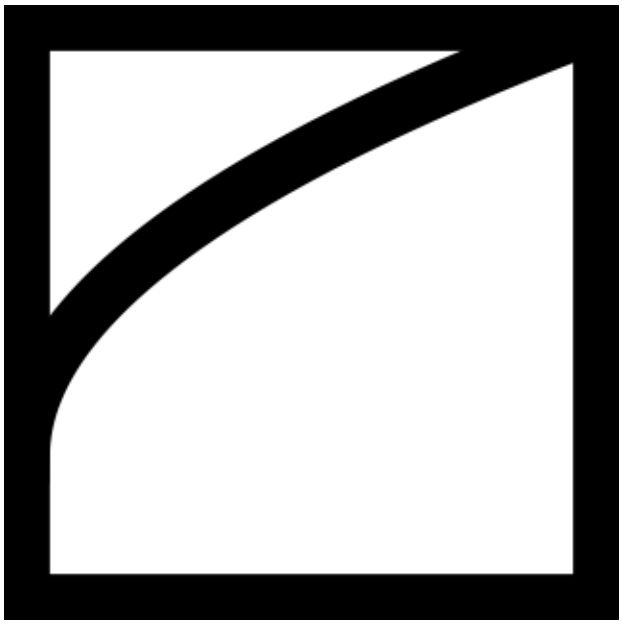
for spine in ax.spines.values():
    spine.set_visible(False)

width = 20

ax.plot(frame_path_x, frame_path_y, color="black", lw=width, clip_on=False,
        solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

ax.plot(y, x, color="black", lw=width)
ax.set(xlim=[0,1],
       ylim=[0,1],
       xticks=[],
       yticks=[],)

fig.savefig("logo_favicon_hires.png", dpi=600, transparent=True, bbox_inches="tight")
```



15.6 Duotone

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)

for spine in ax.spines.values():
    spine.set_visible(False)

width = 10
solid_capstyle = 'round' # 'butt', 'round', 'projecting'
solid_joinstyle = 'round' # 'miter', 'round', 'bevel'

frame_path_x = np.array([0, 1, 1, 0, 0])
frame_path_y = np.array([0, 0, 1, 1, 0])
# ax.plot(frame_path_x, frame_path_y, color="black", lw=width, clip_on=False,
#         solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

color_dark = np.array([24,43,72]) / 255
color_light = np.array([153,163,177]) / 255
# ax.plot(y, x, color="black", lw=width)
ax.fill_between(y, x, 0, color=color_dark, edgecolor="white", linewidth=width)
ax.fill_between(y, x, 1, color=color_light, edgecolor="white", linewidth=width)

ax.set(xlim=[0,1],
       ylim=[0,1],
       xticks=[],
       yticks=[],)
```



15.7 Variation on the theme

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)

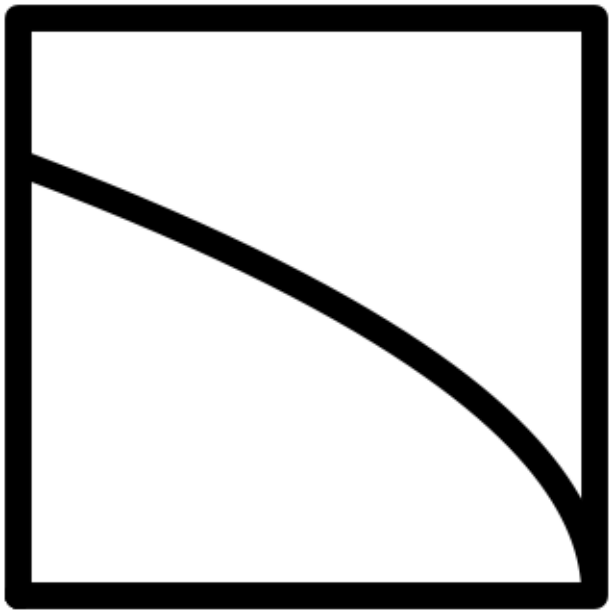
for spine in ax.spines.values():
    spine.set_visible(False)

width = 10
solid_capstyle = 'round' # 'butt', 'round', 'projecting'
solid_joinstyle = 'round' # 'miter', 'round', 'bevel'

ax.plot(frame_path_x, frame_path_y, color="black", lw=width, clip_on=False,
        solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

ax.plot(1-y, x-x0, color="black", lw=width)
ax.set(xlim=[0,1],
       ylim=[0,1],
```

```
       xticks=[],
       yticks=[],)
```



15.8 logo with URL

```
fig=plt.figure(figsize=(5, 5))
fig.subplots_adjust(left=0.0, right=1.0, top=1.0, bottom=0.0,
                    hspace=0, wspace=0)
ax = plt.Axes(fig, [0.2, 0.2, 0.6, 0.6])
fig.add_axes(ax)

for spine in ax.spines.values():
    spine.set_visible(False)

width = 10
solid_capstyle = 'round' # 'butt', 'round', 'projecting'
solid_joinstyle = 'round' # 'miter', 'round', 'bevel'

frame_path_x = np.array([0, 1, 1, 0, 0])
frame_path_y = np.array([0, 0, 1, 1, 0])
ax.plot(frame_path_x, frame_path_y, color="black", lw=width, clip_on=False,
```

```
        solid_capstyle=solid_capstyle, solid_joinstyle=solid_joinstyle)

ax.plot(y, x, color="black", lw=width)
ax.set(xlim=[0,1],
       ylim=[0,1],
       xticks=[],
       yticks=[],)

# Reset font-related rcParams to defaults
mpl.rcParams.update(mpl.rcParamsDefault)

font_path = os.path.expanduser("~/Library/Fonts/Ubuntu-Bold.ttf")
ubuntu_font = font_manager.FontProperties(fname=font_path)

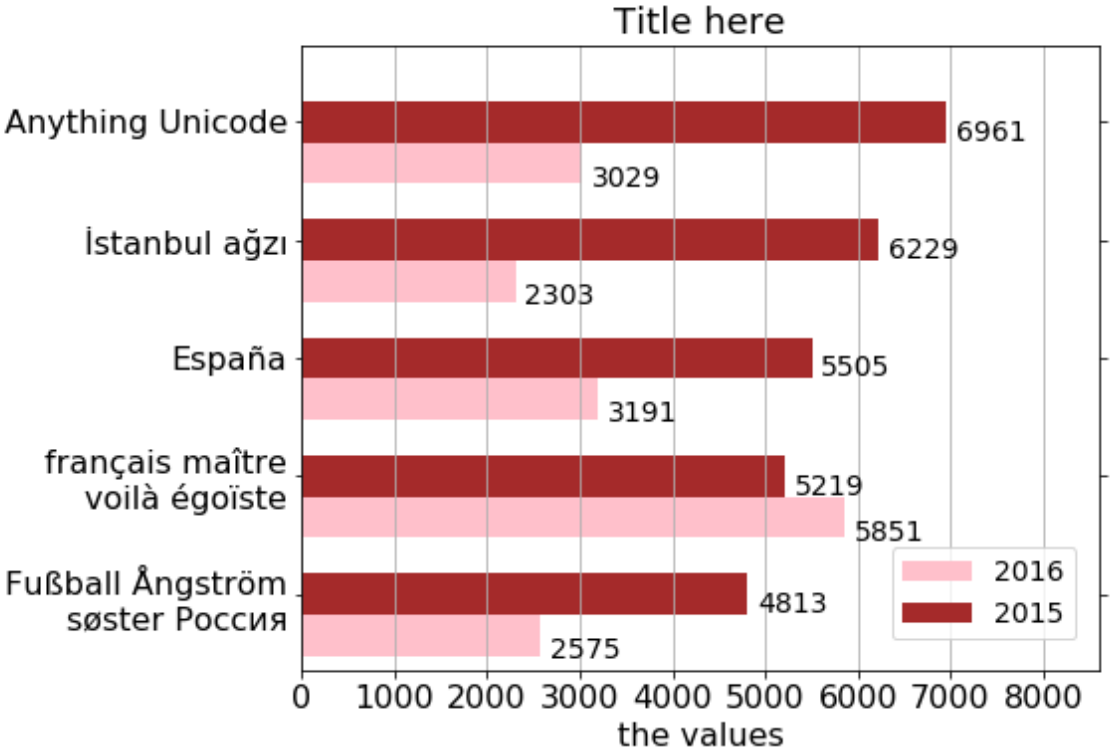
ax.text(0.5, 0.1, 'yairmau.com', transform=ax.transAxes, ha='center', fontsize=32, fontprope

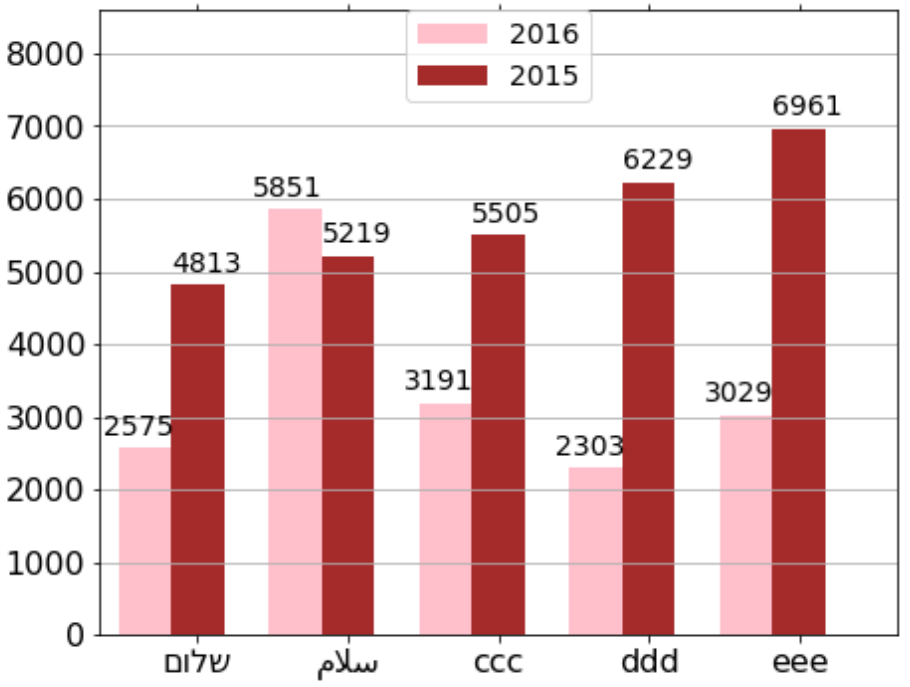
fig.savefig("logo_transparent_bg_URL.png", dpi=600, transparent=True, bbox_inches="tight")
```



16 Bar plot

Vertical and horizontal bar plots


Title here
Anything Unicode
İstanbul ağzı
España
français maître voilà égoïste
Fußball Ångström søster Россия
6961
3029
6229
2303
5505
3191
5219
5851
4813
2575
2016
2015
0
1000
2000
3000
4000
5000
6000
7000
8000
the values


2016
2015
8000
7000
6000
5000
4000
3000
2000
1000
0
2575
4813
5851
5219
3191
5505
2303
6229
3029
6961
שלום
سلام
ccc
ddd
eee

16.1 Introduction

This code produces the figure above. Here we showcase the use of unicode text.

16.2 The code

```
from __future__ import unicode_literals
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from bidi import algorithm as bidialg      # needed for arabic, hebrew
import arabic_reshaper                     # needed for arabic
```

```
ModuleNotFoundError: No module named 'arabic_reshaper'
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
<ipython-input-2-aaa74850711b> in <module>
      4 import matplotlib.pyplot as plt
      5 from bidi import algorithm as bidialg      # needed for arabic, hebrew
----> 6 import arabic_reshaper                     # needed for arabic

ModuleNotFoundError: No module named 'arabic_reshaper'
```

```
# http://wiki.scipy.org/Cookbook/Matplotlib/LaTeX_Examples
pts_per_inch = 72.27       # this is a latex constant, don't change it.
text_width_in_pts = 300.0  # write "\the\textwidth" (or "\showthe\columnwidth" for a 2 collu
                           # inside a figure environment in latex, the result will be on the
text_width_in_inches=text_width_in_pts/pts_per_inch
golden_ratio = 0.618     # make rectangles with a nice proportion
inverse_latex_scale = 2  # figure.png or figure.eps will be intentionally larger, because it
                         # when compiling latex code, use \includegraphics[scale=(1/inverse_l
fig_proportion = (3.0 / 3.0) # we want the figure to occupy 2/3 (for example) of the text wi
csize = inverse_latex_scale * fig_proportion * text_width_in_inches
fig_size = (1 * csize, 1.3 * csize)  # always 1.0 on the first argument
text_size = inverse_latex_scale * 10  # find out the fontsize of your latex text, and put it
label_size = inverse_latex_scale * 10
tick_size = inverse_latex_scale * 8
# learn how to configure:
# http://matplotlib.sourceforge.net/users/customizing.html
params = {'backend': 'ps',
```

```
          'axes.labelsize': 16,
          'legend.fontsize': 14,
          'legend.handlelength': 2.5,
          'legend.borderaxespad': 0,
          'axes.labelsize': label_size,
          'xtick.labelsize': tick_size,
          'ytick.labelsize': tick_size,
          # 'font.family': 'serif',
          'font.size': text_size,
          # 'font.serif': ['Computer Modern Roman'],
          'ps.usedistiller': 'xpdf',
          # 'text.usetex': True,
          'figure.figsize': fig_size,
          #'text.latex.unicode': True,
          }
plt.rcParams.update(params)

plt.ioff()
fig = plt.figure(1, figsize=fig_size)  # figsize accepts only inches.
fig.clf()

dpi = 100
pixel_size = (700,700)
fig_size = (pixel_size[0]/dpi,pixel_size[1]/dpi)
ax1=fig.add_subplot(211)
ax2=fig.add_subplot(212)
fig.subplots_adjust(left=0.30, right=0.97, top=0.95, bottom=0.06,
                    hspace=0.2, wspace=0.1)
```

```
words = [u'Fußball Ångström\nsøster    ',
         u'français maître\nvoilà égoïste',
         u'España',
         u'İstanbul ağzı',
         u'Anything Unicode'
         ]
values1 = [2575, 5851, 3191, 2303, 3029]
values2 = [4813, 5219, 5505, 6229, 6961]
values1 = np.array(values1)
values2 = np.array(values2)
width = 0.35       # the width of the bars
r = np.arange(len(values1))
```

ax1 , horizontal bars

```
v1 = ax1.barh(r, values1, width, color='pink')
v2 = ax1.barh(r + width, values2, width, color='brown')
ax1.axis([0, 8600, r.min() - 0.3, r.max() + 1])
ax1.set_yticks(r)
ax1.set_yticks(r + 1 * width)
ax1.set_yticklabels(words)
xt = np.arange(0, 8100, 1000)
ax1.set_xticks(xt)
ax1.set_xticklabels(xt)
ax1.set_xlabel(u'the values', fontsize=16)
ax1.set_title(u'Title here', fontsize=18)
ax1.xaxis.grid(True)

ax1.tick_params(
    axis='y',          # changes apply to the y-axis
    which='both',      # both major and minor ticks are affected
    left='off',        # ticks along the left edge are off
    right='off',       # ticks along the right edge are off
    labelleft='on')    # labels along the bottom edge are on

ax1.legend((v1, v2),
          (u'2016', u'2015'),
          loc=(0.74,0.05))

def autolabel_hor(rects,ax, offset_x, offset_y):
    # attach some text labels at the tip of the bars
    for i,rect in enumerate(rects):
        width = rect.get_width()
        height = rect.get_height()
        ax.text(width + offset_x, rect.get_y() + offset_y * height,
                '%d' % int(width),
                ha='left', va='bottom', fontsize=14)
autolabel_hor(v1, ax1, 100.0, -0.20)
autolabel_hor(v2, ax1, 100.0, -0.10)
```

ax2, vertical bars

```
from bidi import algorithm as bidialg

w1 = ax2.bar(r, values1, width, color='pink')
```

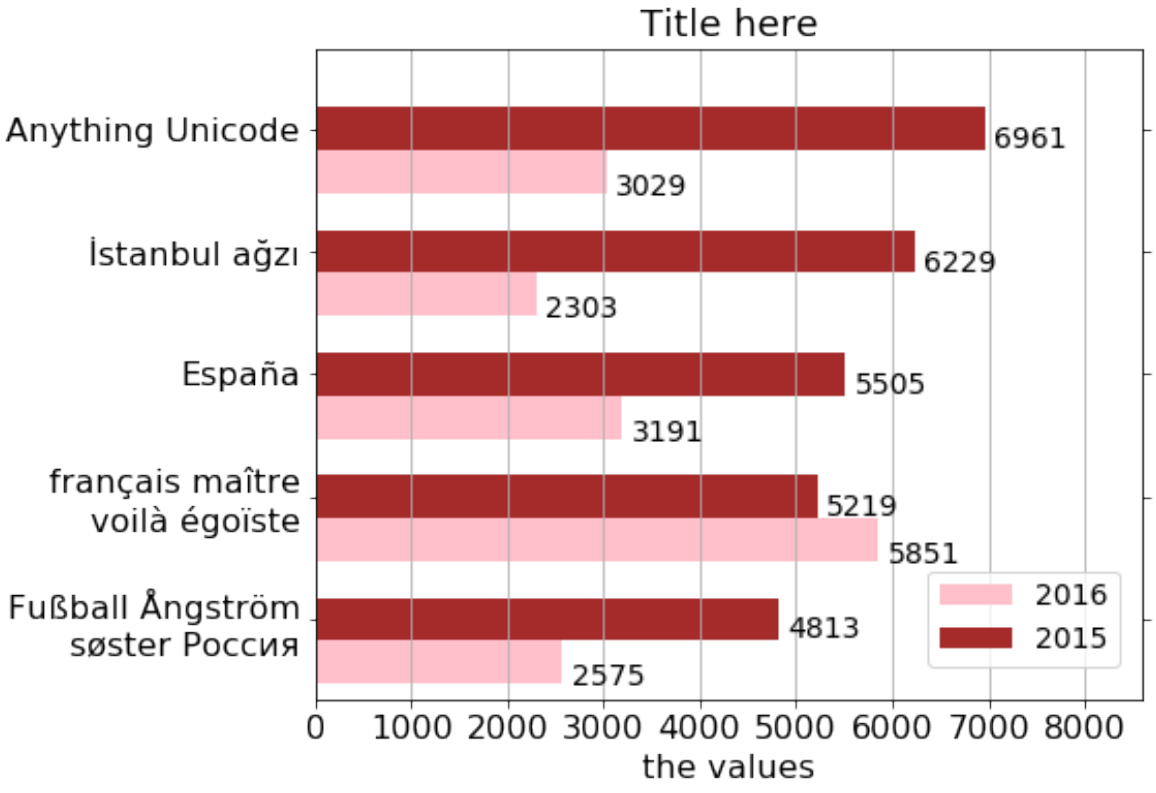
```
w2 = ax2.bar(r + width, values2, width, color='brown')
ax2.axis([r.min() - 0.3, r.max() + 1, 0, 8600])
ax2.set_xticks(r)
ax2.set_xticks(r + 1 * width)
shalom = bidialg.get_display(u'  ')
salam = bidialg.get_display(arabic_reshaper.reshape(u'  '))
ax2.set_xticklabels([shalom, salam, 'ccc', 'ddd', 'eee'])
xt = np.arange(0, 8200, 1000)
ax2.set_yticks(xt)
ax2.set_yticklabels(xt)
ax2.yaxis.grid(True)

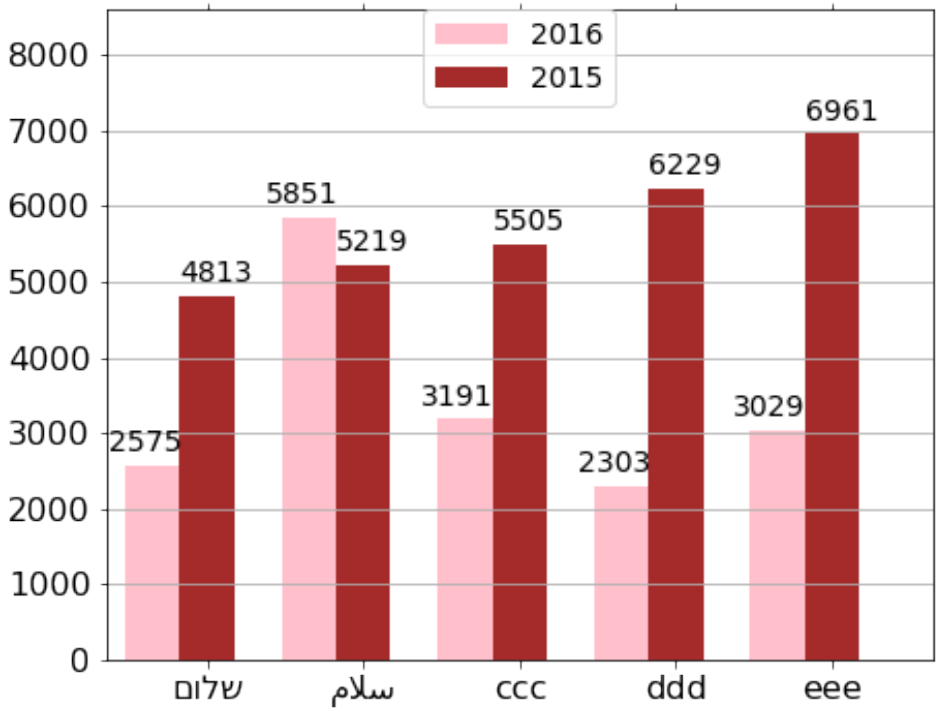
ax2.tick_params(
    axis='x',          # changes apply to the x-axis
    which='both',      # both major and minor ticks are affected
    top='off',         # ticks along the top edge are off
    bottom='off',      # ticks along the bottom edge are off
    labelbottom='on')  # labels along the bottom edge are on

ax2.legend((w1, w2),
          (u'2016', u'2015'),
          loc="upper center")

def autolabel_ver(rects,ax, offset_x, offset_y):
    # attach some text labels at the tip of the bars
    for i,rect in enumerate(rects):
        width = rect.get_width()
        height = rect.get_height()
        ax.text(rect.get_x() + offset_x * width, height + offset_y,
                '%d' % int(height),
                ha='left', va='bottom', fontsize=14)
autolabel_ver(w1, ax2, -0.3, 100.0)
autolabel_ver(w2, ax2, 0., 100.0)
```

```
%matplotlib inline
fig.savefig("./python_figures/bars.png")
plt.show()
fig
```


Title here
Anything Unicode
İstanbul ağzı
España
français maître voilà égoïste
Fußball Ångström søster Россия
6961
3029
6229
2303
5505
3191
5219
5851
4813
2575
2016
2015
0
1000
2000
3000
4000
5000
6000
7000
8000
the values


2016
2015
8000
7000
6000
5000
4000
3000
2000
1000
0
2575
4813
5851
5219
3191
5505
2303
6229
3029
6961
שלום
سلام
ccc
ddd
eee